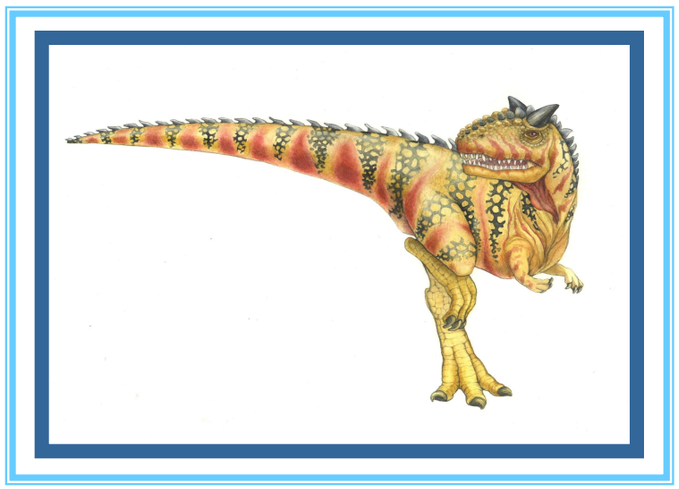
Chapter 8: Memory-Management Strategies



Silberschatz, Galvin and Gagne ©2013



Chapter 8: Memory Management Strategies

- 8.1 Background
- 8.2 Swapping
- 8.3 Contiguous Memory Allocation
- 8.4 Segmentation
- 8.5 Paging

- OBJECTIVES
- To provide a detailed description of various ways of organizing memory hardware

- To discuss various memory-management techniques, including paging and segmentation





Background

- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly
- Memory unit only sees a stream of addresses + read requests, or address + data and write requests
- Register access in one CPU clock (or less)
- Main memory can take many cycles, causing a **stall**
- **Cache** sits between main memory and CPU registers
- Protection of memory required to ensure correct operation





Base and Limit Registers

- A pair of **base** and **limit registers** define the logical address space
- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user

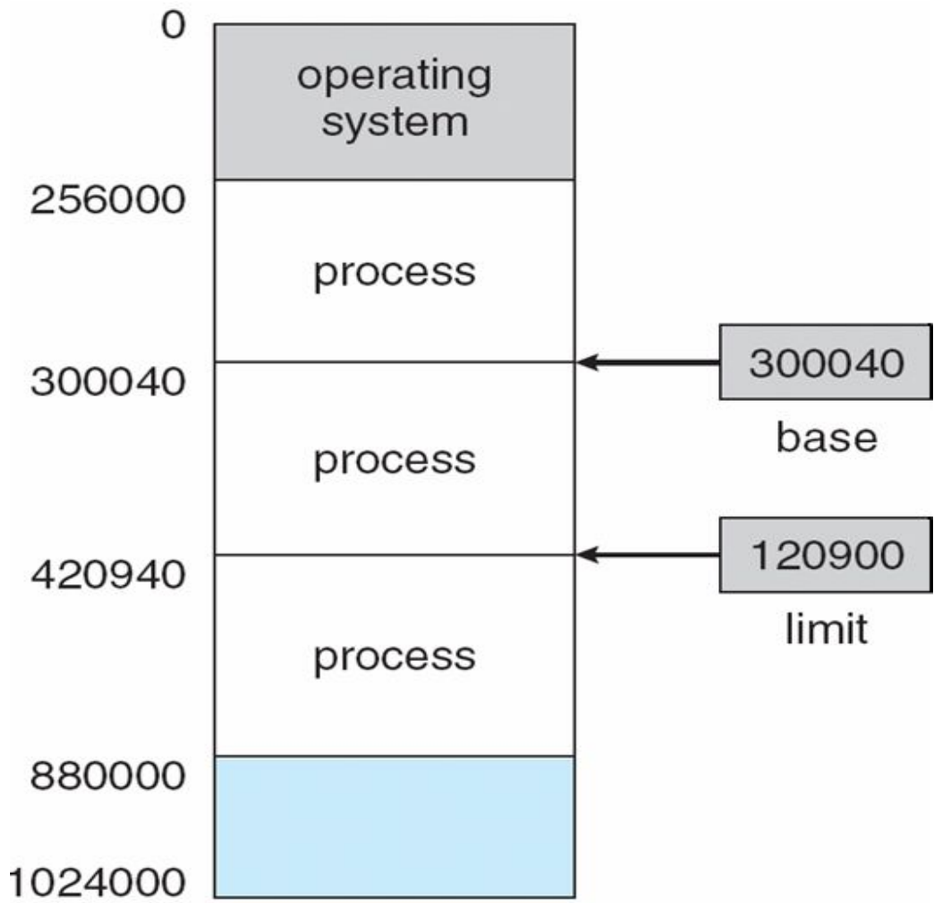


Figure 8.1 A base and a limit register define a logical address space.





Logical vs. Physical Address Space

- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
  - **Logical address** – generated by the CPU; also referred to as **virtual address**
  - **Physical address** – address seen by the memory unit

- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program





Memory-Management Unit (MMU)

- Hardware device that at run time maps virtual to physical address
- Many methods possible, covered in the rest of this chapter
- To start, consider simple scheme where the value in the relocation register is added to every address generated by a user process at the time it is sent to memory
  - Base register now called **relocation register**
  - MS-DOS on Intel 80x86 used 4 relocation registers
- The user program deals with *logical* addresses; it never sees the *real* physical addresses
  - Execution-time binding occurs when reference is made to location in memory
  - Logical address bound to physical addresses



- The user program deals with *logical addresses*; it never sees the ***real*** *physical addresses*.

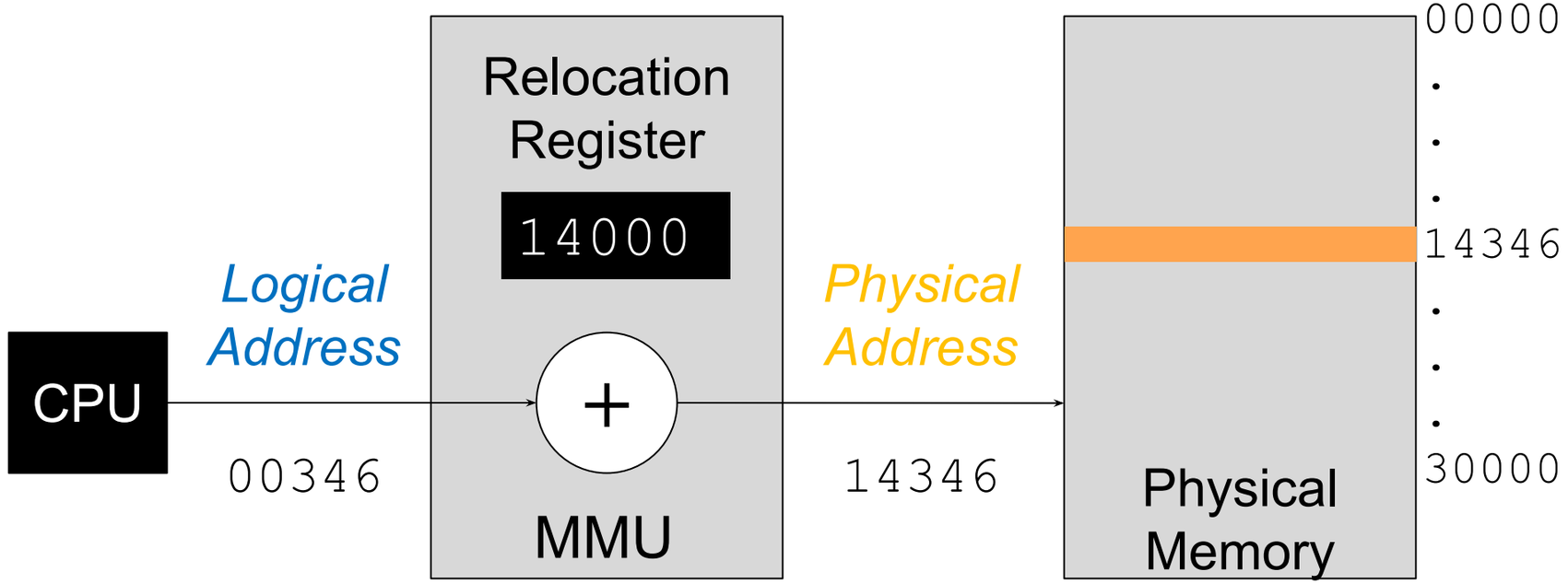


Figure 8.4

The concept of a **logical address space** (bound to separate **physical address space**) is central to proper memory management.



Swapping

- A process can be **swapped** temporarily out of memory to a backing store, and then brought back into memory for continued execution
  - Total physical memory space of processes can exceed physical memory
- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk
- Does the swapped out process need to swap back in to same physical addresses?
- Depends on address binding method
  - Plus consider pending I/O to / from process memory space
- Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows)
  - Swapping normally disabled
  - Started if more than threshold amount of memory allocated
  - Disabled again once memory demand reduced below threshold



Schematic View of Swapping

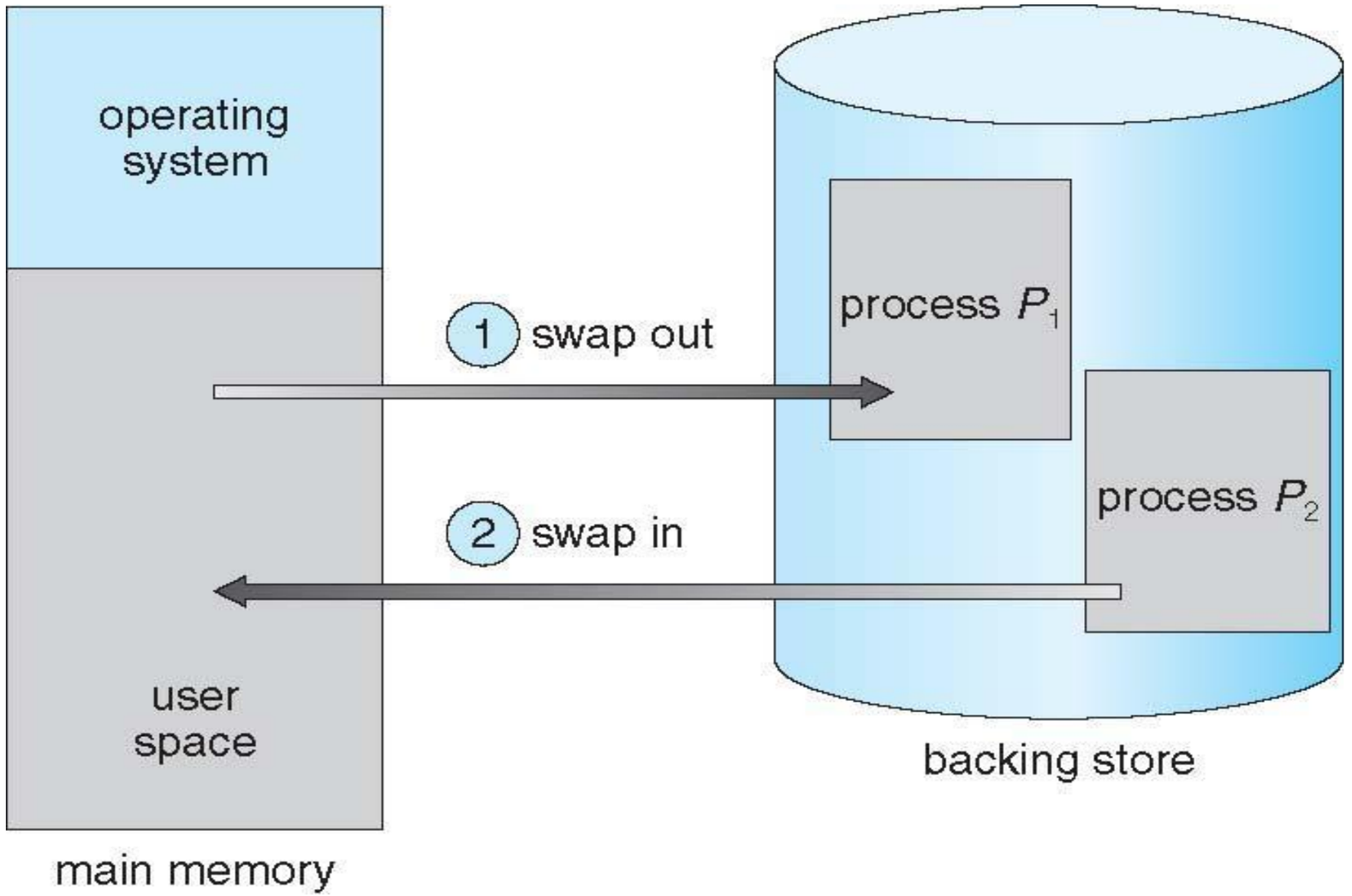


Figure 8.5 Swapping of two processes using a disk as a backing store.



Context Switch Time including Swapping

- If next processes to be put on CPU is not in memory, need to swap out a process and swap in target process
- Context switch time can then be very high
- 100MB process swapping to hard disk with transfer rate of 50MB/sec
  - Swap out time of 2000 ms
  - Plus swap in of same sized process
  - Total context switch swapping component time of 4000ms (4 seconds)
- Can reduce if reduce size of memory swapped – by knowing how much memory really being used
  - System calls to inform OS of memory use via `request memory()` and `release memory()`
- Other constraints as well on swapping
  - Pending I/O – can't swap out as I/O would occur to wrong process
  - Or always transfer I/O to kernel space, then to I/O device
    - Known as **double buffering**, adds overhead
- Standard swapping not used in modern operating systems
  - But modified version common
    - Swap only when free memory extremely low





Swapping on Mobile Systems

- Not typically supported
  - Flash memory based
    - Small amount of space
    - Limited number of write cycles
    - Poor throughput between flash memory and CPU on mobile platform
- Instead use other methods to free memory if low
  - iOS ***asks*** apps to voluntarily relinquish allocated memory
    - Read-only data thrown out and reloaded from flash if needed
    - Failure to free can result in termination
  - Android terminates apps if low free memory, but first writes **application state** to flash for fast restart
  - Both OSes support paging as discussed below



Memory Allocation

Contiguous

– each process is contained in a single contiguous section of memory

Fixed Partitions

Dynamic/Variable Partitions

Non-Contiguous

– each process is contained in a non-sequential location of memory

Paging

Segmentation



Contiguous Memory Allocation

Contiguous
– sharing a common border.
– next or together in sequence.

- Main memory must support both OS and user processes
- Limited resource, must allocate efficiently
- Contiguous allocation is one early method

- Main memory usually into two **partitions**:
  - Resident operating system, usually held in low memory with interrupt vector
  - User processes then held in high memory
  - Each process contained in single contiguous section of memory





(a) Fixed-sized Partition : Example

- Each partition may contain exactly one process.
- Degree of multiprogramming limited by number of partitions
- Multiple-partition allocation

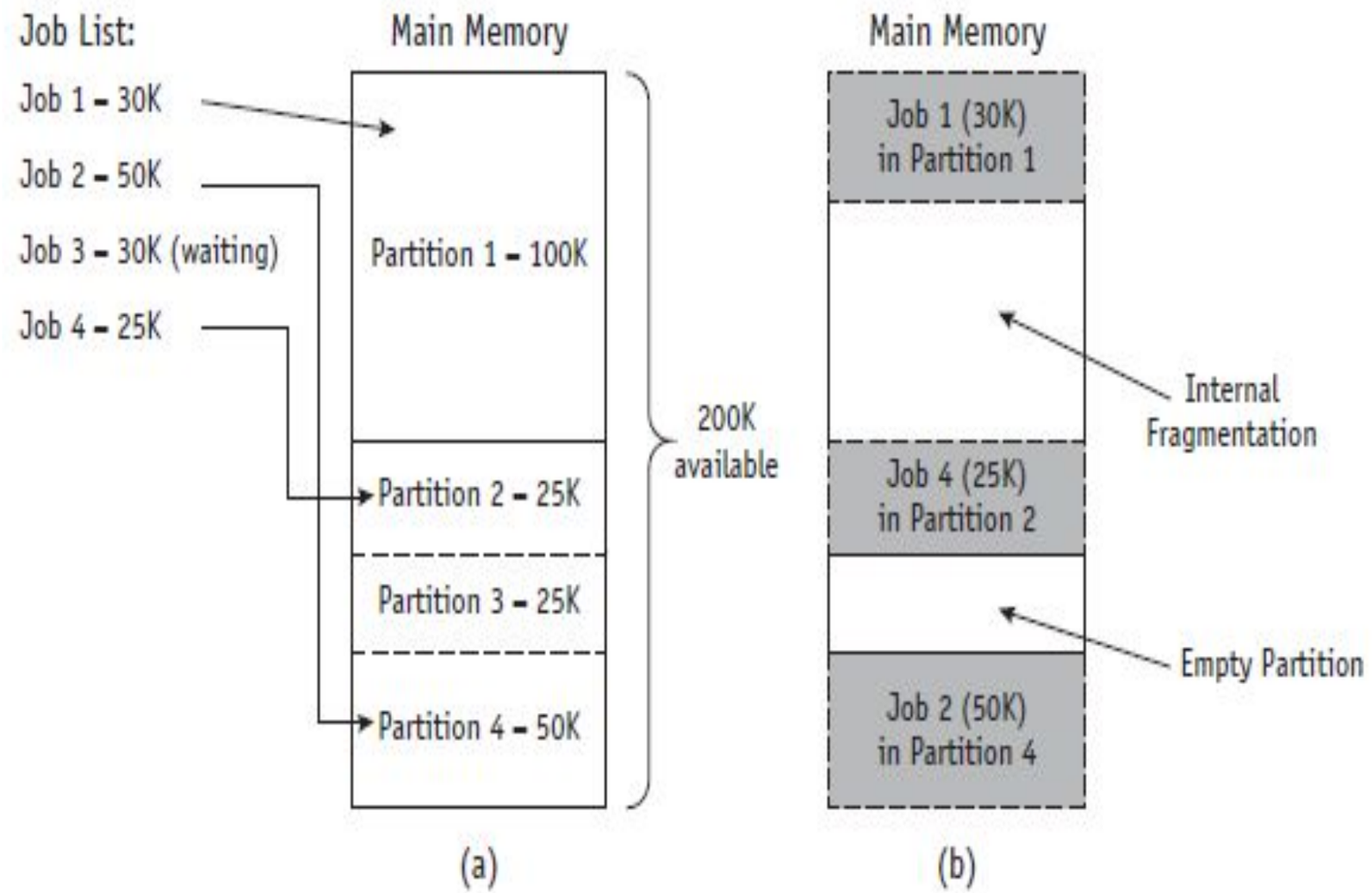
- **Fixed-sized partitions**
  - Equal-size partitions & unequal-size partitions

Table A simplified fixed-partition memory table with the free partition shaded.

Partition Size	Memory Address	Access	Partition Status
100K	200K	Job 1	Busy
25K	300K	Job 4	Busy
25K	325K		Free
50K	350K	Job 2	Busy



Contiguous Memory Allocation (Cont..)



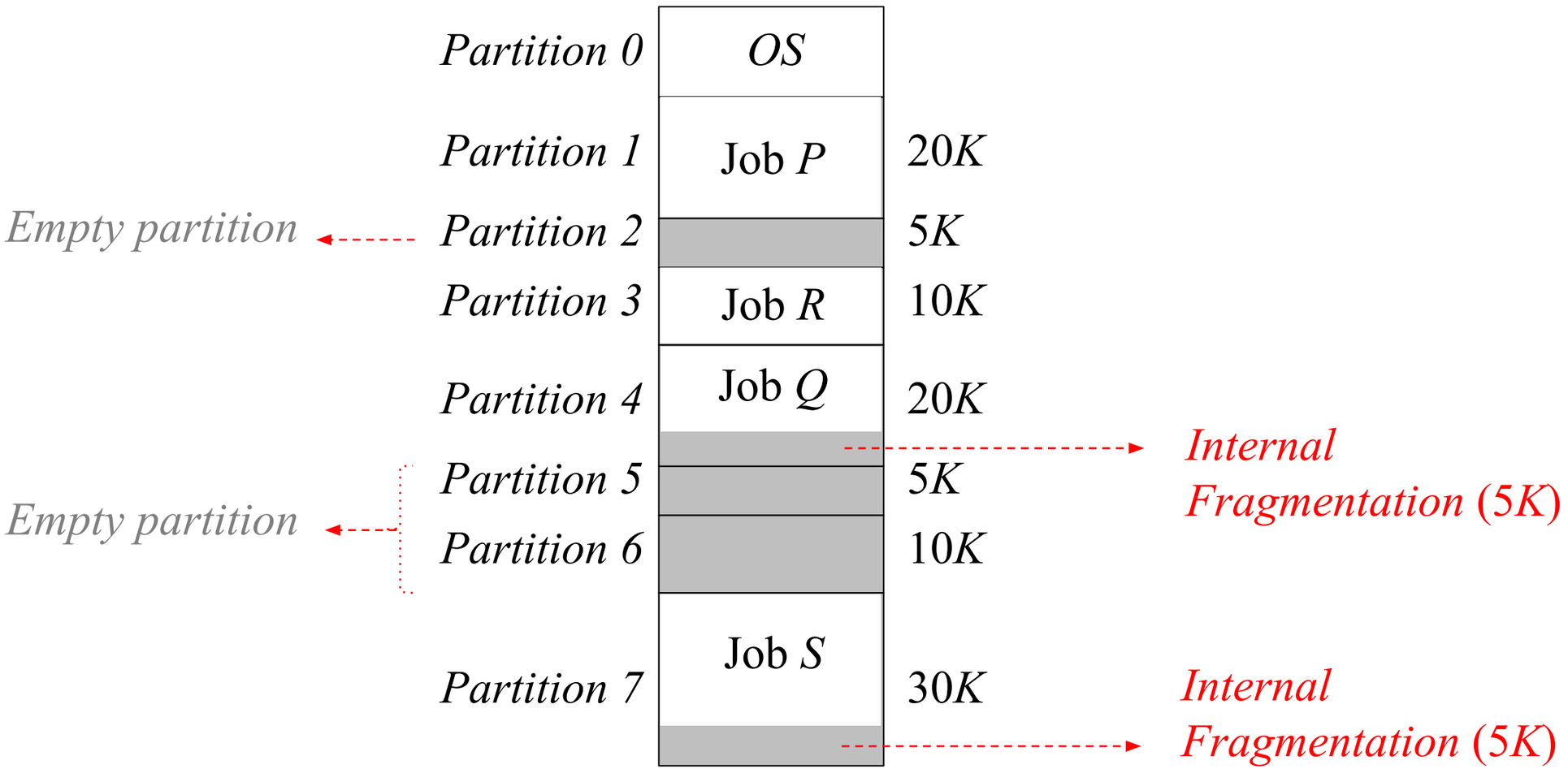
(figure 2.3)

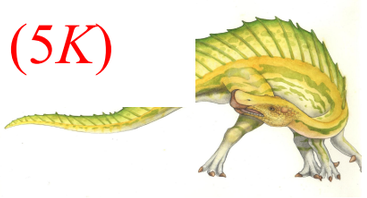
Main memory use during fixed partition allocation of Table 2.1. Job 3 must wait even though 70K of free space is available in Partition 1, where Job 1 only occupies 30K of the 100K available. The jobs are allocated space on the basis of "first available partition of required size."



Example 1

The new jobs in the order: Job P (20K) – Job Q (15K) – Job R (10K) – Job S (25K) need to be allocated in a **fixed partition** memory as shown in the figure. Label all the jobs in which partition each will be allocated.







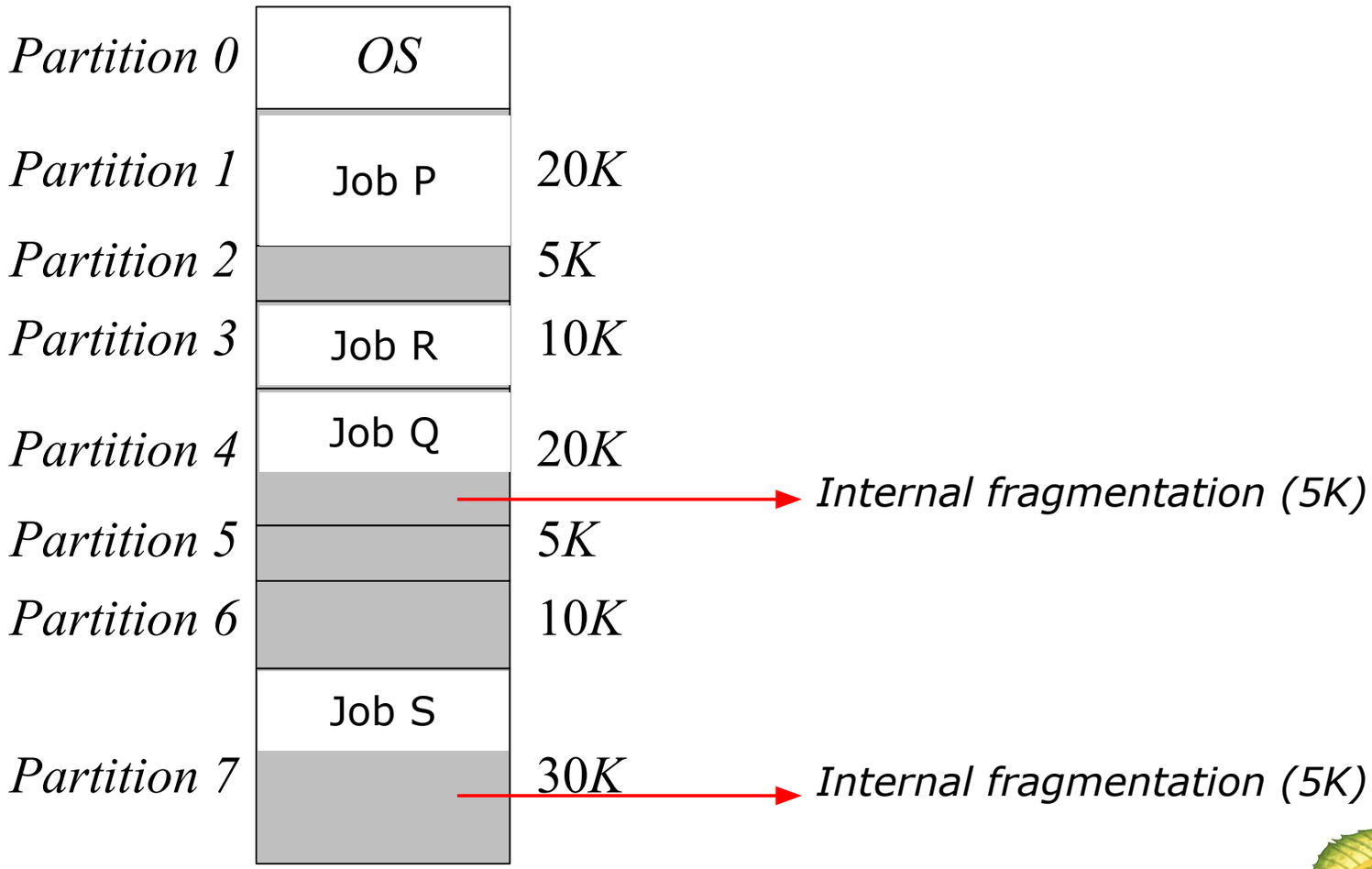
Solution

Fixed partition: Job P (20K)

Job Q (15K)

Job R (10K)

Job S (25K)







Contiguous Memory Allocation (Cont..)

- **Disadvantages Fixed Partitions**
  - Requires contiguous loading of entire program.
  - Job allocation method:
    - *First available partition with required size.*
  - Arbitrary partition size leads to undesired results:

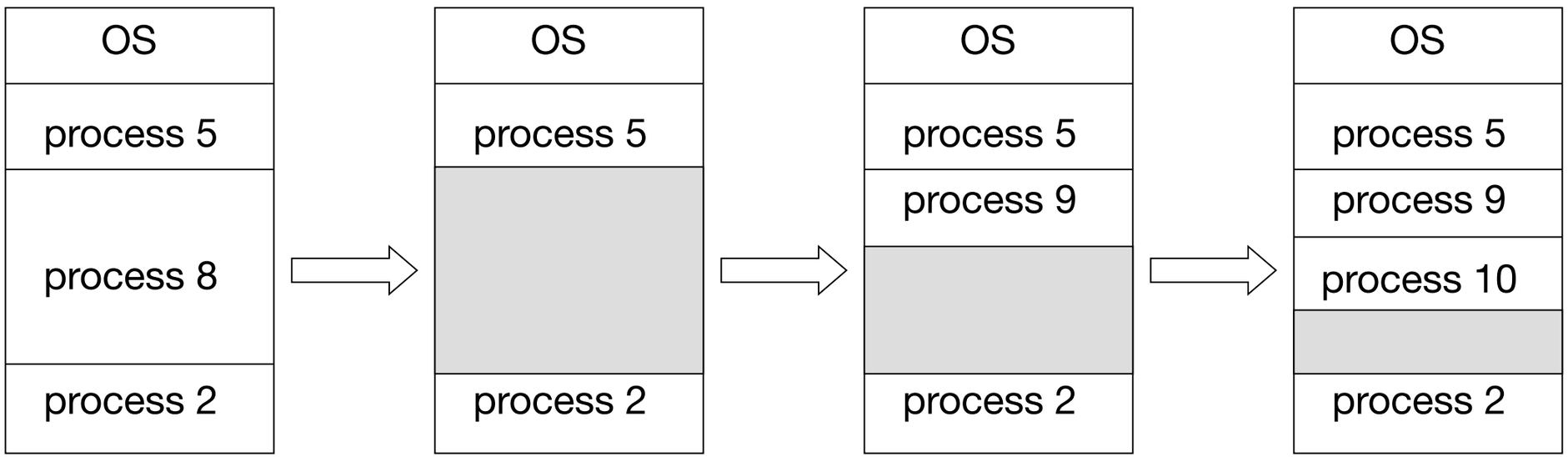
▪ Partition too small □ Large jobs have longer turnaround time.
▪ Partition too large □ Memory waste: internal fragmentation.





(b) Dynamic/Variable Partition

- **Dynamic/Variable-partition** sizes for efficiency (sized to a given process' needs)
  - **Hole** – block of available memory; holes of various size are scattered throughout memory
  - When a process arrives, it is allocated memory from a hole large enough to accommodate it
  - Process exiting frees its partition, adjacent free partitions combined
  - Operating system maintains information about:
    a) allocated partitions b) free partitions (hole)







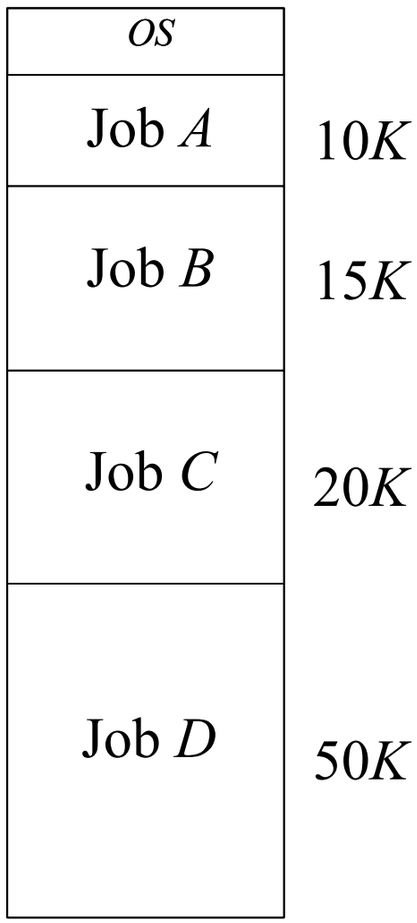
Example 2

Variable/Dynamic partition: Five snapshots (a – e) of main memory as 8 jobs submitted and allocated space on the basis "*First-Come, First-Serve*".

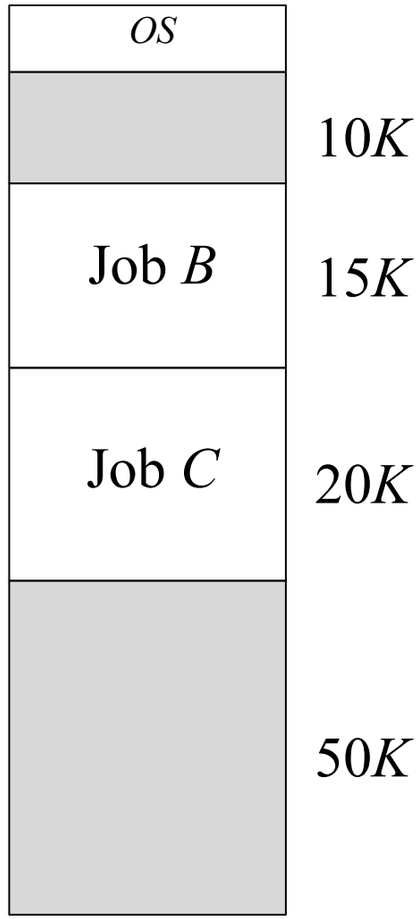
Init. jobs:
A (10*K*)
B (15*K*)
C (20*K*)
D (50*K*)

New jobs:
E (5*K*)
F (30*K*)

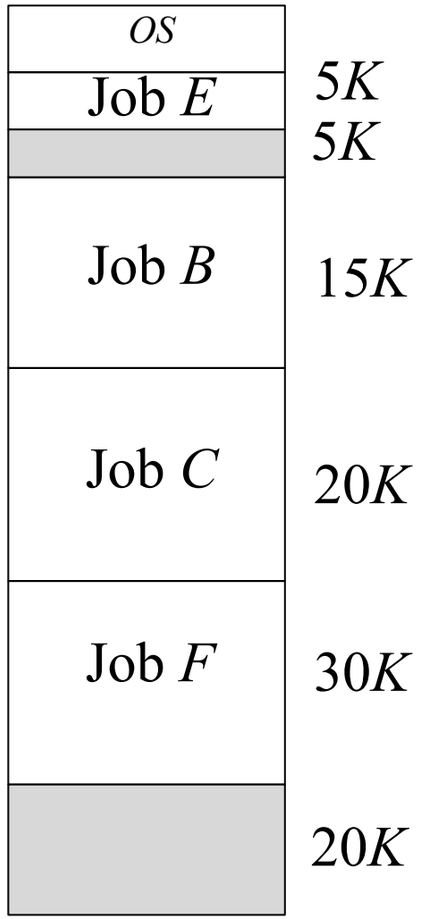
New jobs:
G (10*K*)
H (30*K*)



(a)
Initial job entry memory allocation.



(b)
After Job *A* and *D* have finished.



(c)
After Job *E* and *F* have entered.

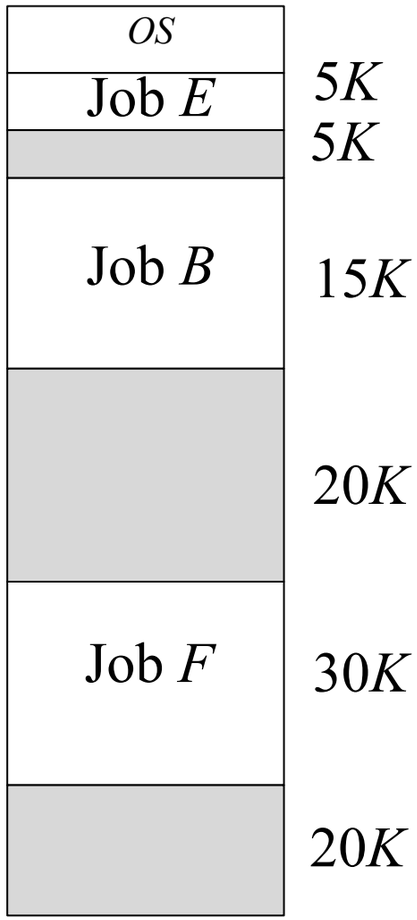


Contiguous Memory Allocation (Cont..)

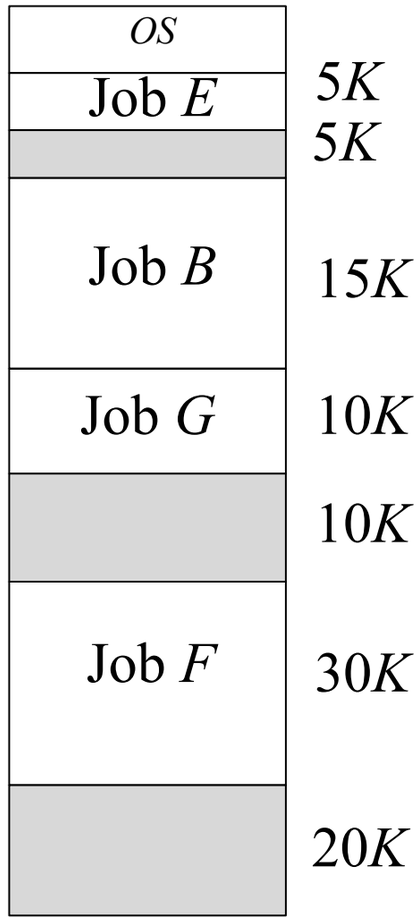
Init. jobs:
A (10*K*)
B (15*K*)
C (20*K*)
D (50*K*)

New jobs:
E (5*K*)
F (30*K*)

New jobs:
G (10*K*)
H (30*K*)



(d)
After Job *C* has finished.



(e)
After Job *G* has entered.

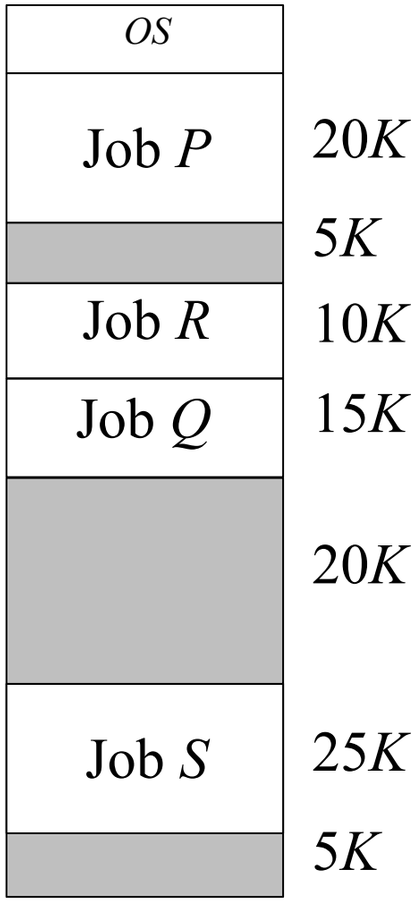
Job *H* has to wait even though there's enough free memory between partition to accommodate it.



Exercise 2

Figure shows a few jobs have been allocated in a **dynamic partition** memory at time t_0. Suppose that job P and Q finished at t_1 and new jobs arrived at t_3. Allocate all the new jobs in the memory.

New jobs:
T (30K)
U (35K)
V (15K)





Solution

t_0: Initial jobs in memory.

t_1: After Job P and Q have finished.

t_3: After Job T and V have entered.

Job U need to wait.

New jobs:
T (30K)
U (35K)
V (15K)





- Based on **examples before** Job H and U respectively have to wait until any job ends with enough size.

- **<u>Disadvantages</u>**:

  Produced through block of available memory (**holes**) with different sizes scattered out the memory.

- **<u>Solution</u>**:

  Relocatable Dynamic Partition

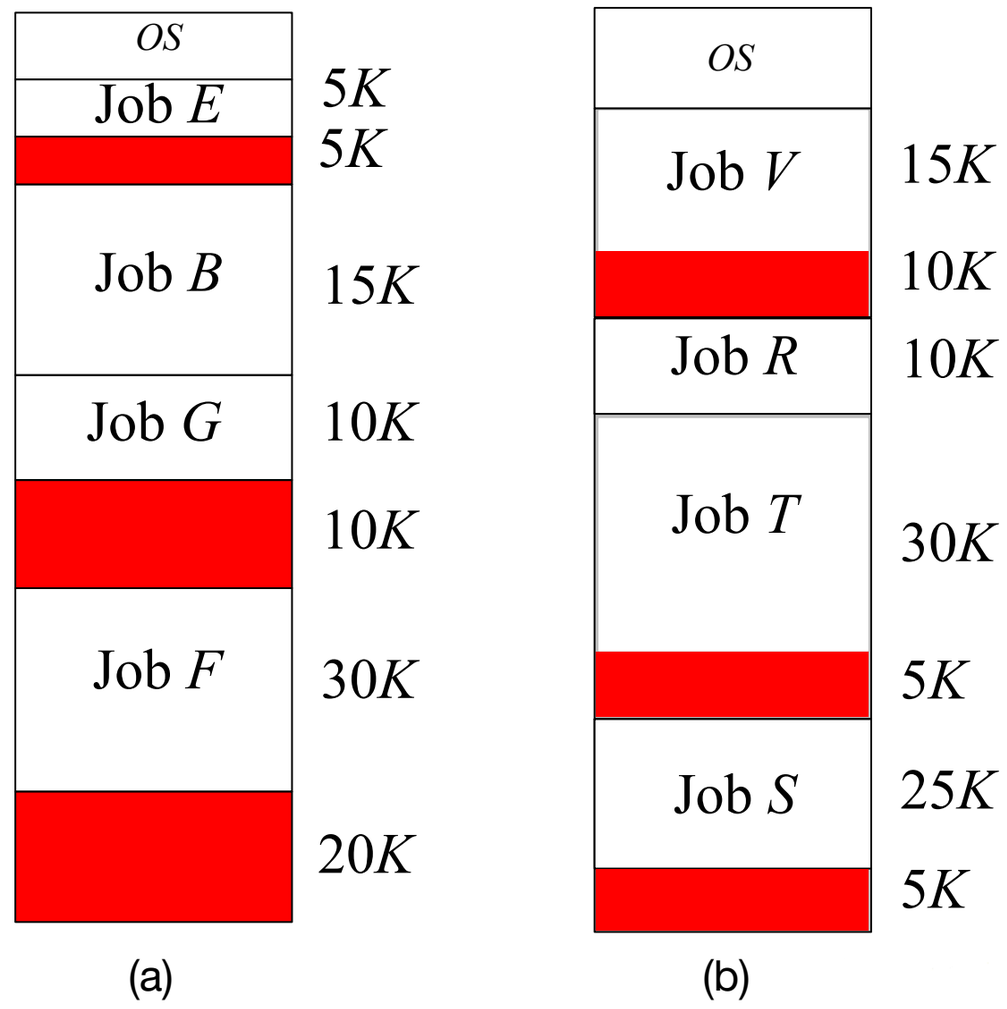
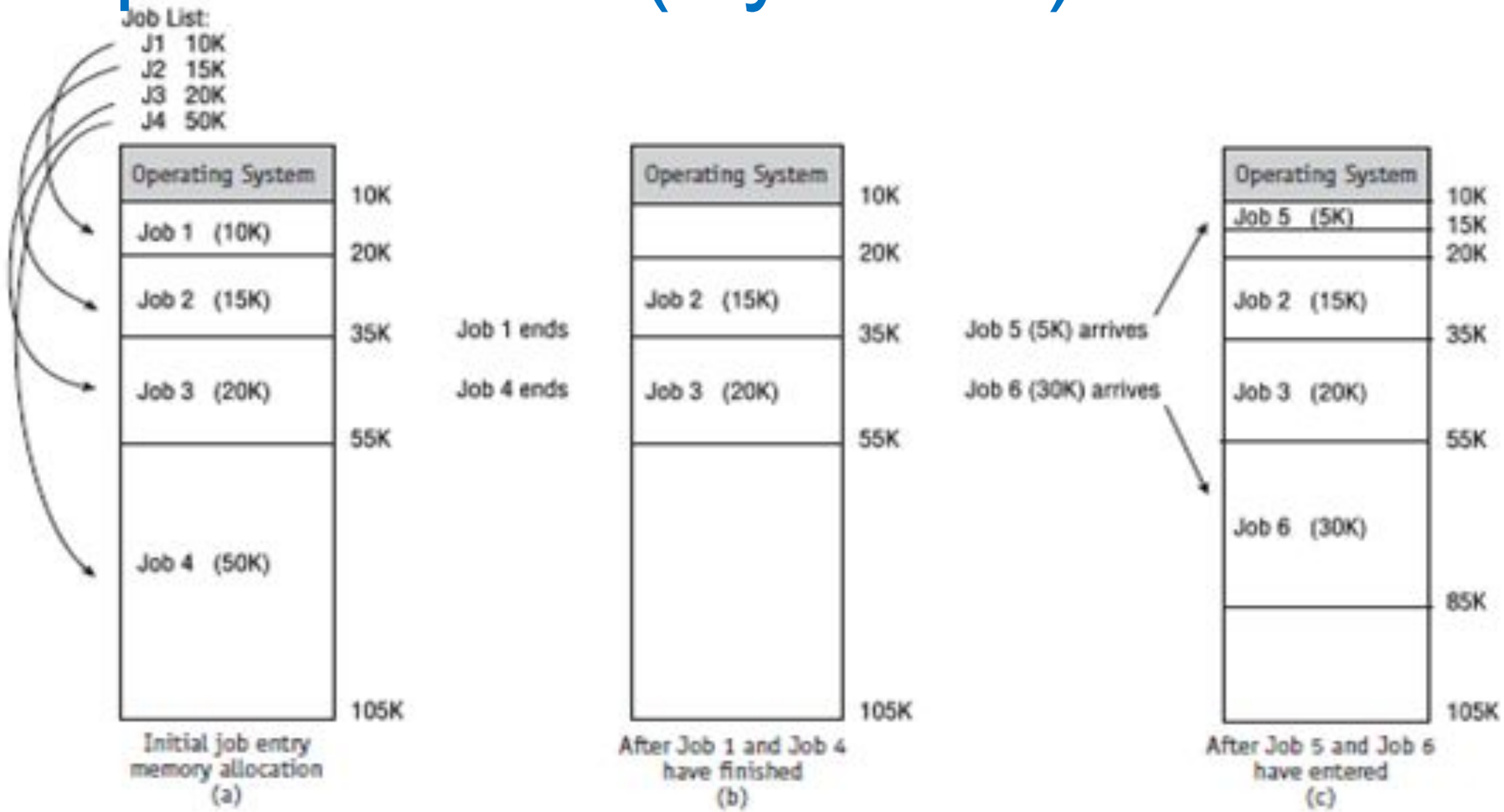


Figure Holes or external fragmentation examples before



Contiguous Memory Allocation (Cont..)

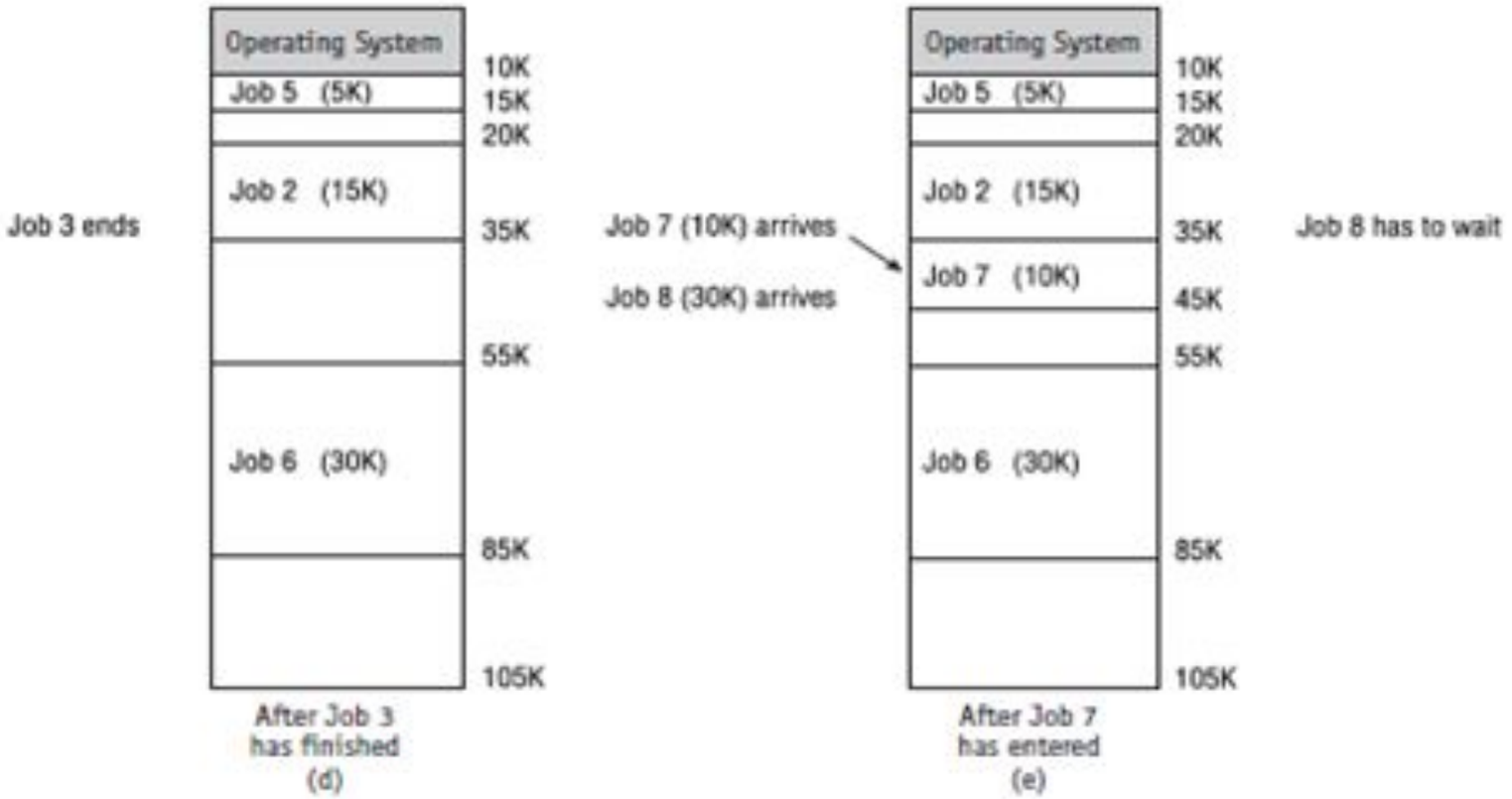
Example Variable (Dynamic) Partitions



(figure 2.4)

Main memory use during dynamic partition allocation. Five snapshots (a-e) of main memory as eight jobs are submitted for processing and allocated space on the basis of "first come, first served." Job 8 has to wait (e) even though there's enough free memory between partitions to accommodate it.

Contiguous Memory Allocation (Cont..)



(figure 2.4)

Main memory use during dynamic partition allocation. Five snapshots (a-e) of main memory as eight jobs are submitted for processing and allocated space on the basis of "first come, first served." Job 8 has to wait (e) even though there's enough free memory between partitions to accommodate it.

Fragmentation

- There are two types of fragmentation:

Internal fragmentation

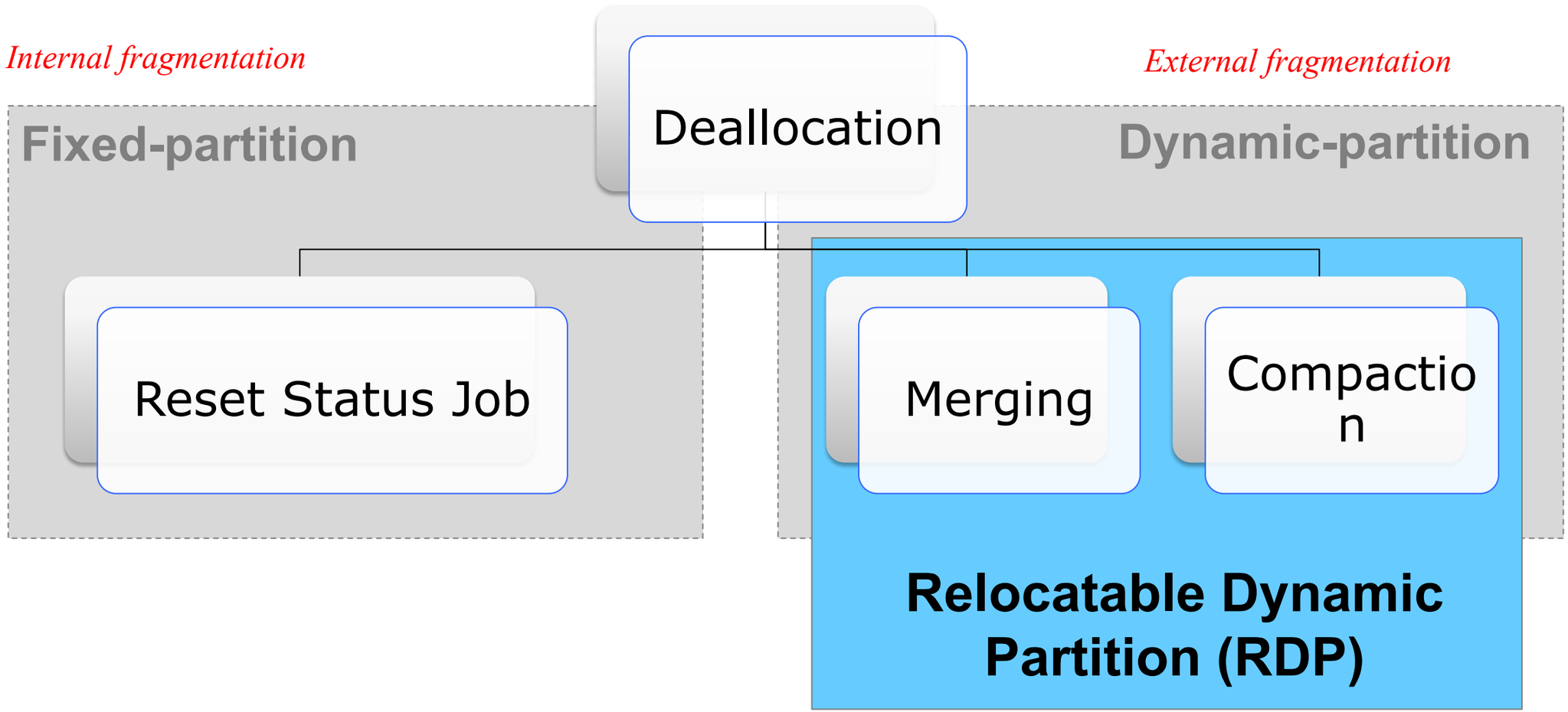
  - allocated memory may be slightly larger than requested memory; the difference of memory internal to a partition that not being used;
  - Results from **Fixed partition allocation**.

External fragmentation

  - total memory space exists to satisfy a request, but it is not contiguous.
  - Results from **Dynamic/Variable partition allocation.**

Deallocation

- Solution to reduce fragmentation
  - **Deallocation** : freeing allocated memory spaces.



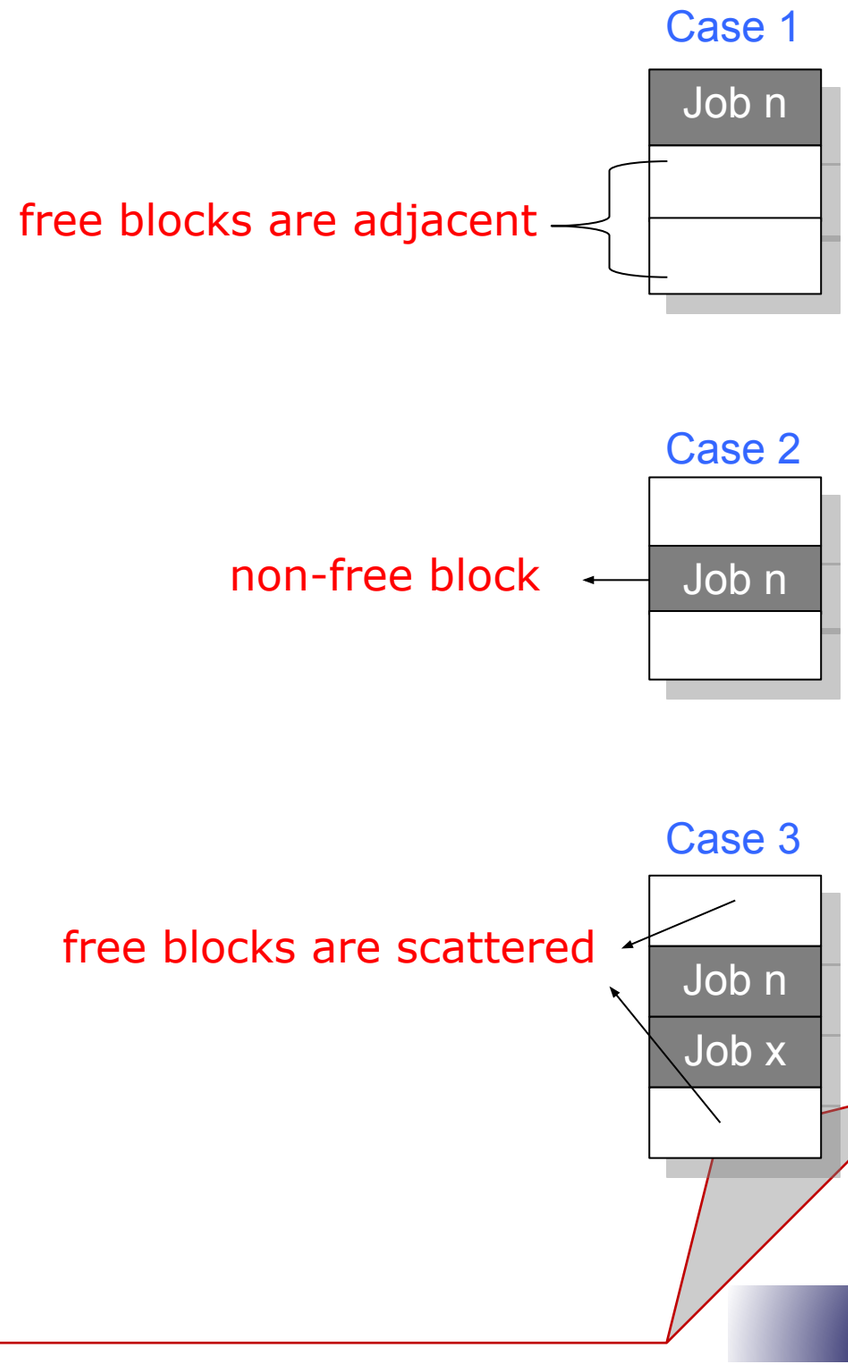
- For fixed-partition system:
  - Straightforward process
  - Memory manager resets the status of job's memory block to free upon job completion
  - Example
    - Binary values with
      - 0 indicating FREE, and 1 indicating BUSY

- For dynamic-partition system:
  - Algorithm tries to merge free areas of memory
  - More complex

- Reduce external fragmentation by **merging and compaction**

  - **Compaction**: reclaiming fragmented sections of memory space. Every program in memory must be relocated.
  - Shuffle all memory contents to one large block.
  - *Happen in* dynamic partition
  - and done at execution time
  - It is called **Relocatable Dynamic Partition**

- Three cases of dynamic partition:
  - **Case 1**: When the free blocks (holes) are adjacent
  - **Case 2:** When the free blocks are separated by a non-free block
  - **Case 3:** When the free blocks are scattered
- Memory Manager relocates programs
  - Merge all empty blocks (Case 1);
  - Compact the empty blocks (Case 2 and 3)



Case 1 – Example 3

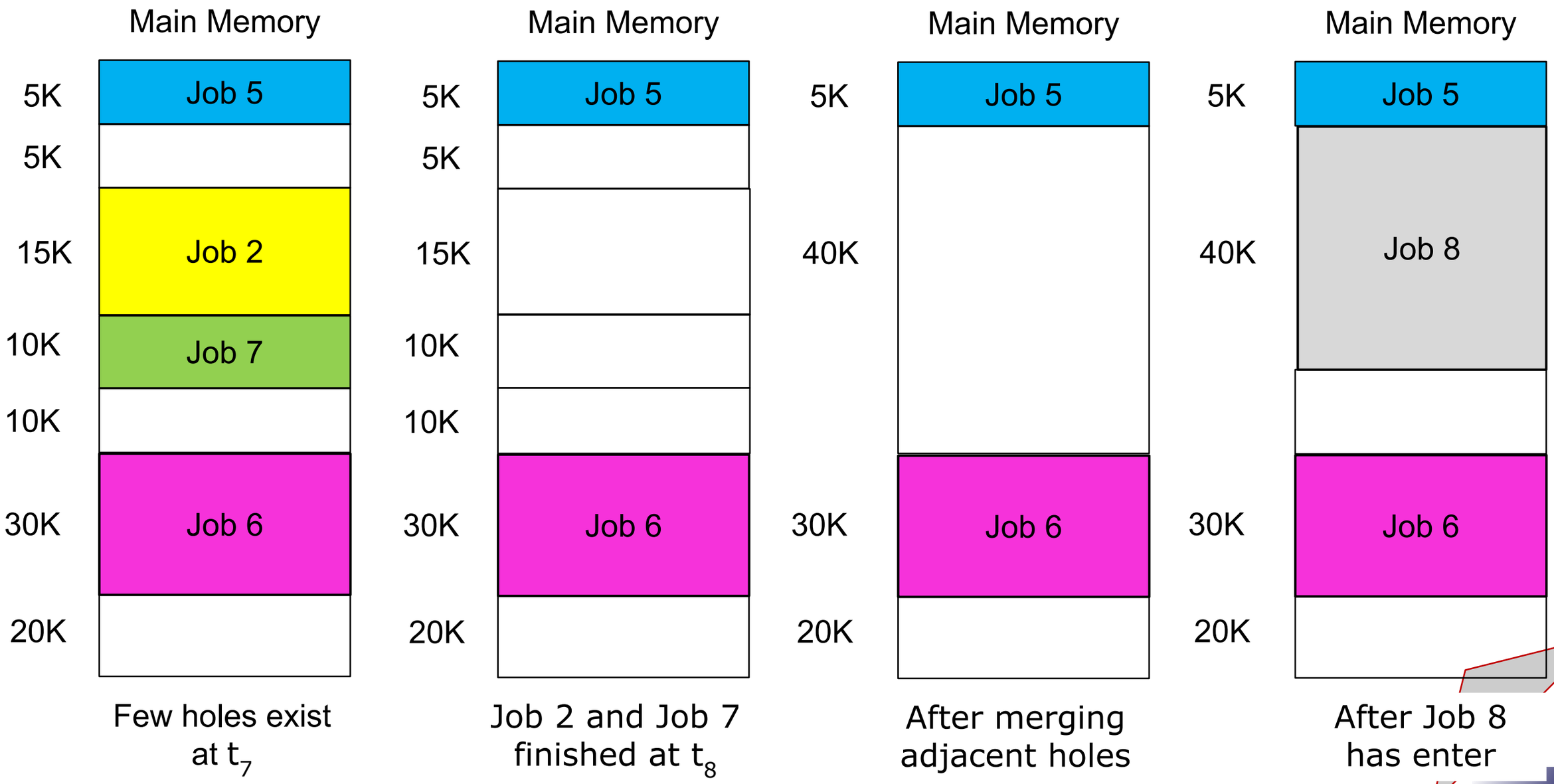


Suppose Job 8 (30K) in waiting state and Job 2 and Job 7 finished at time t_8.

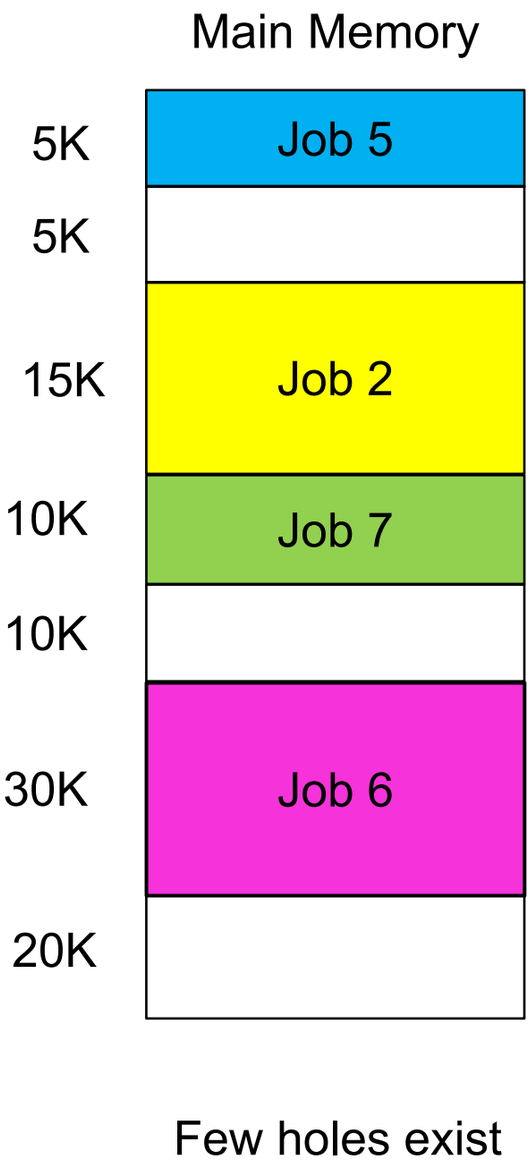
How Job 8 could enter the memory.?

Solution:

Compact (Merge or combine) the adjacent holes to become larger.



Case 2 – Example 3



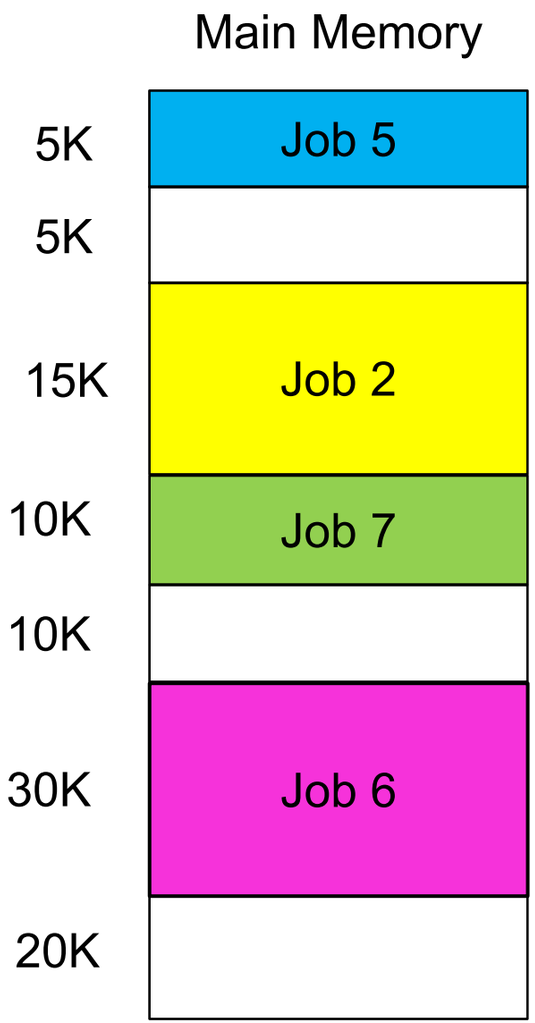
Few holes exist

Suppose Job 8 (30K) in waiting state and Job 2 and Job 7 finished at time t_8.

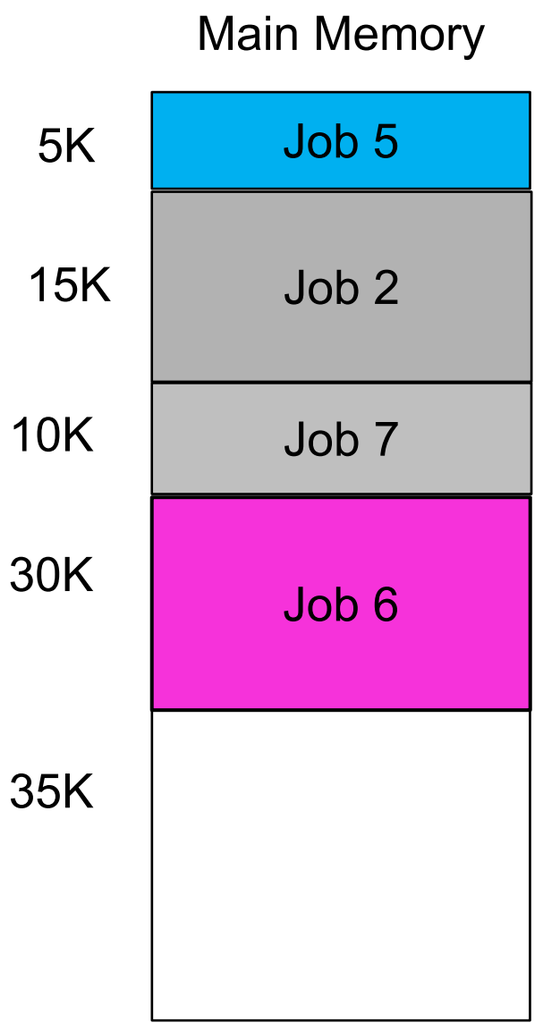
How Job 8 could enter the memory.?

Solution:

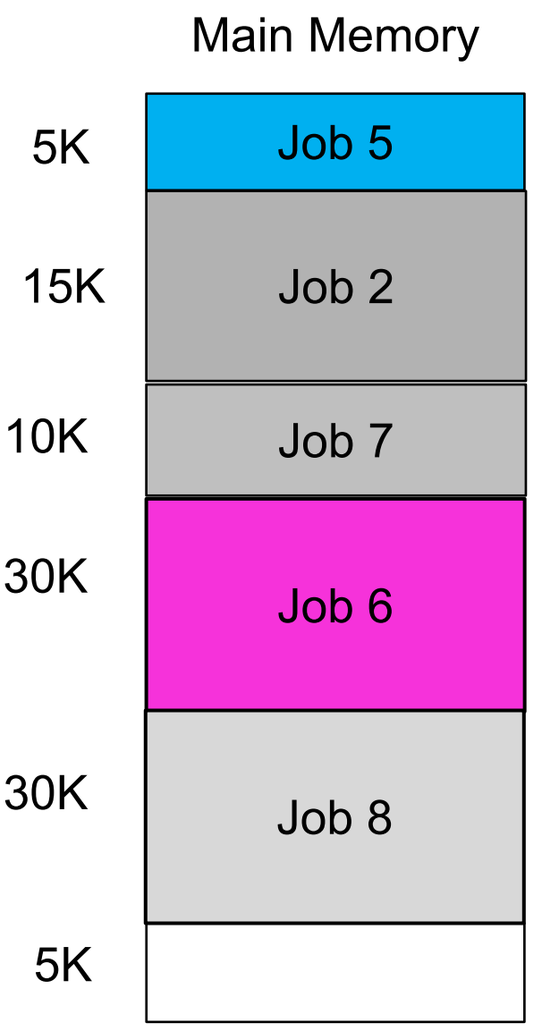
Compact (Merge or combine) all scattered holes at one location.



Few holes exist at t_7



After merging the free holes at t_8



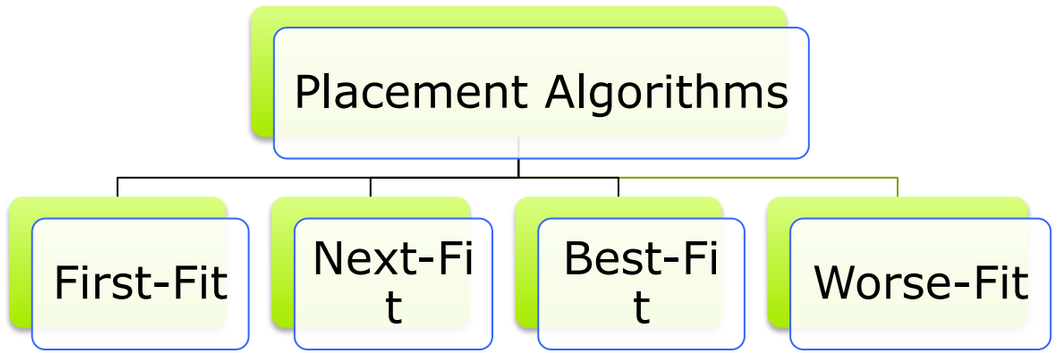
After Job 8 has entered



Dynamic Storage-Allocation Problem

How to satisfy a request of size ***n*** from a list of free holes?

- Use the **Placement Algorithms** for selecting among free blocks of main memory.



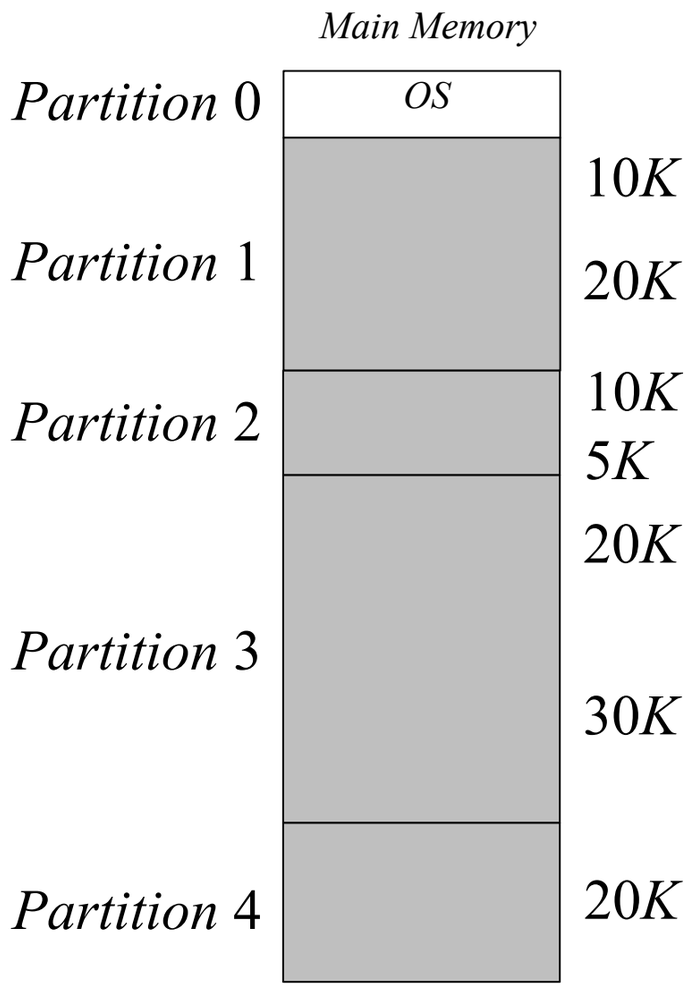
- **First-fit**: Allocate the ***first*** hole that is big enough. Search start at the beginning of memory.
- Next-fit: Similar to first-fit, except the search starts at the last hole allocated.
- **Best-fit**: Allocate the ***smallest*** hole that is big enough; must search entire list, unless ordered by size
  - Produces the smallest leftover hole
- **Worst-fit**: Allocate the ***largest*** hole; must also search entire list
  - Produces the largest leftover hole





Example 4: First-Fit Allocation

(Assume fixed partition are used)



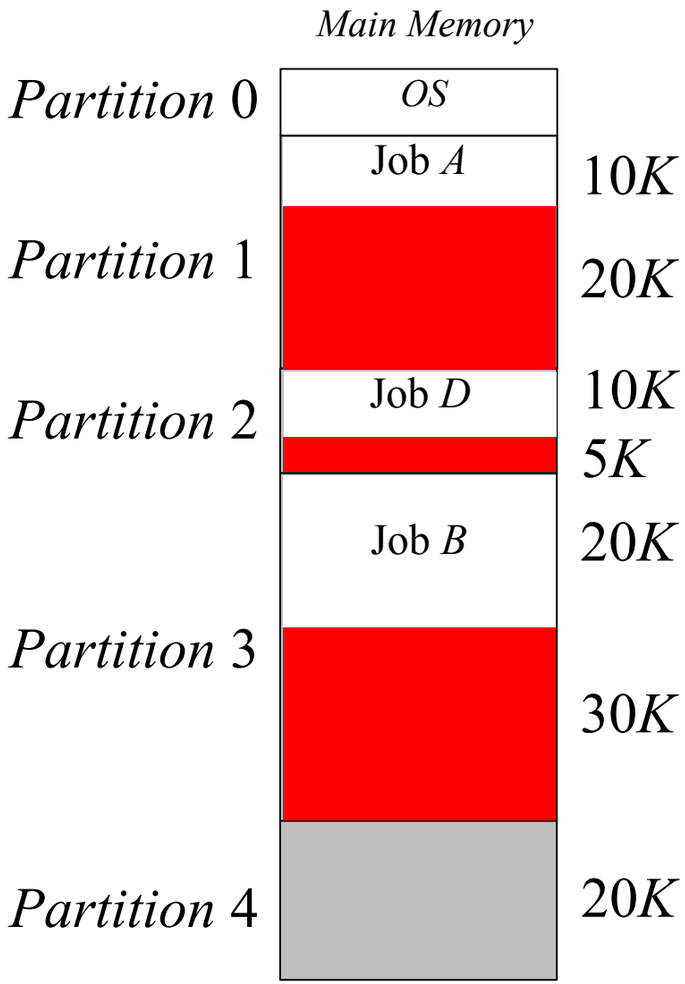
Init. jobs:
A (10*K*)
B (20*K*)
C (30*K*)
D (10*K*)

Unused partition (20*K*)

Internal Fragmentation (55*K*)







Unused partition (20K)

Internal Fragmentation (55K)

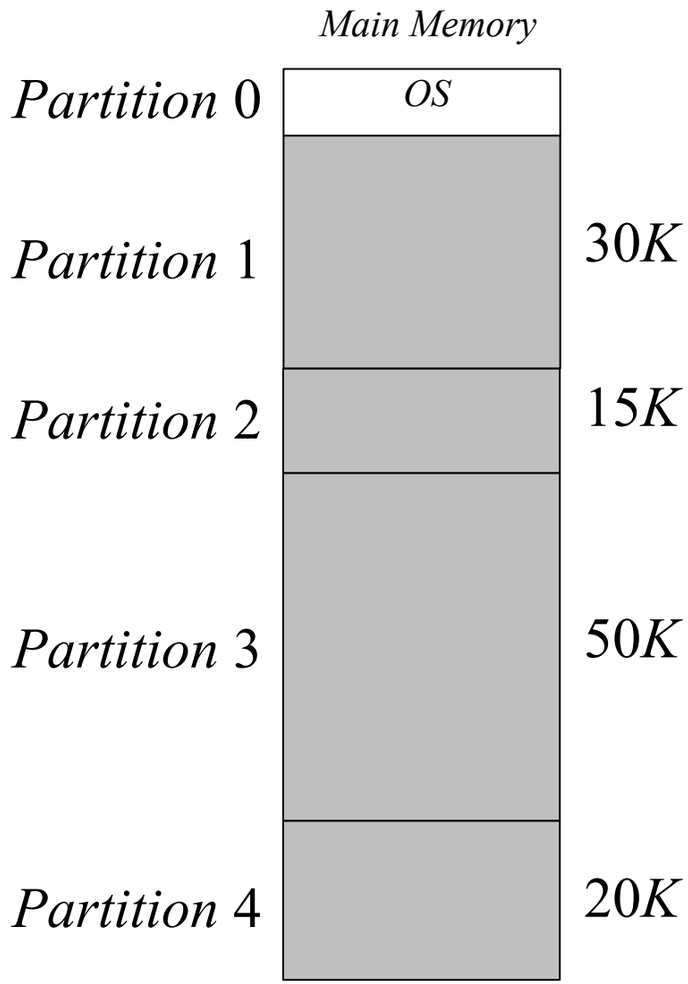
Init. jobs:
A (10K)
B (20K)
C (30K)
D (10K)

- Job C has to wait until a large block become available, even though there is 75K unused memory space.



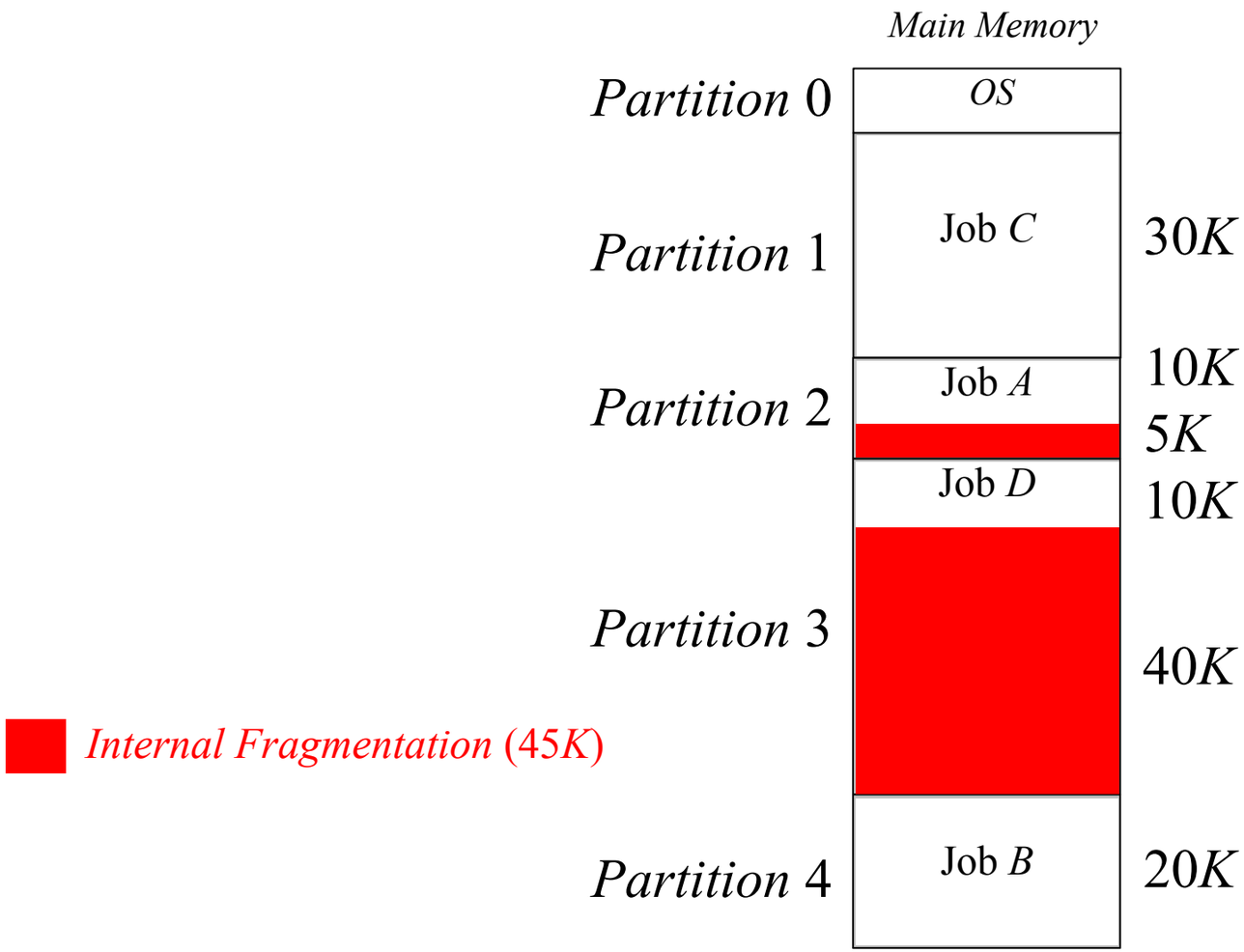
Example 5: Best-Fit Allocation

(Assume fixed partition are used)



Init. jobs:
A (10*K*)
B (20*K*)
C (30*K*)
D (10*K*)

(Assume fixed partition are used)



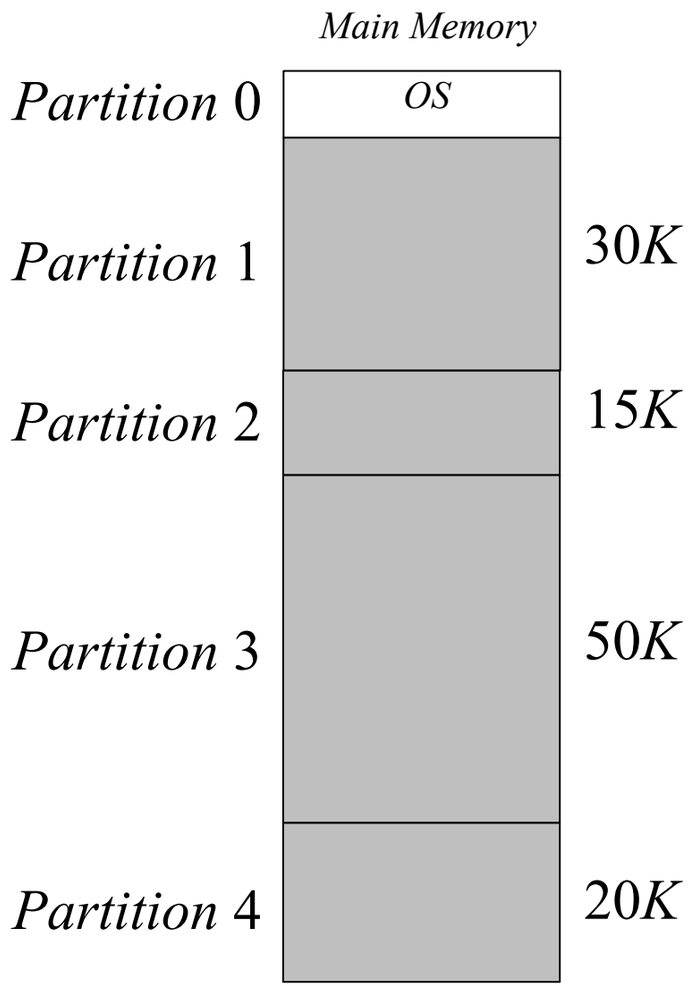
Init. jobs:
- *A* (10*K*)
- *B* (20*K*)
- *C* (30*K*)
- *D* (10*K*)

- ✔ All jobs can be allocated in the memory.
- ✔ Use the memory efficiently but slower implementation.



Example 6: Next-Fit Allocation

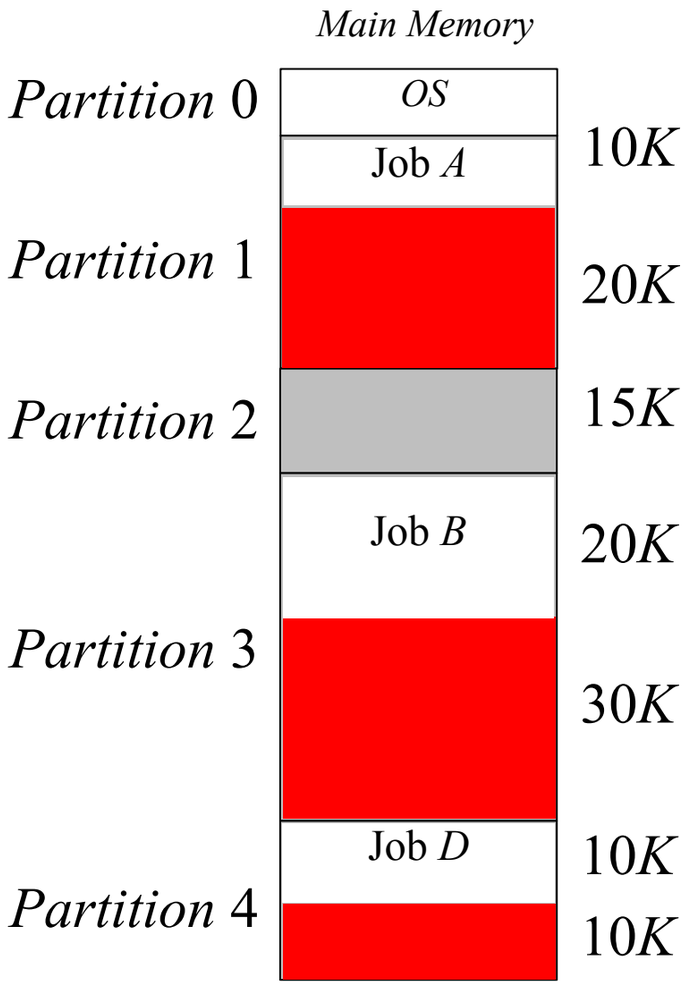
(Assume fixed partition are used)



Init. jobs:
A (10*K*)
B (20*K*)
C (30*K*)
D (10*K*)



(Assume fixed partition are used)



Init. jobs:
A (10*K*)
B (20*K*)
C (30*K*)
D (10*K*)

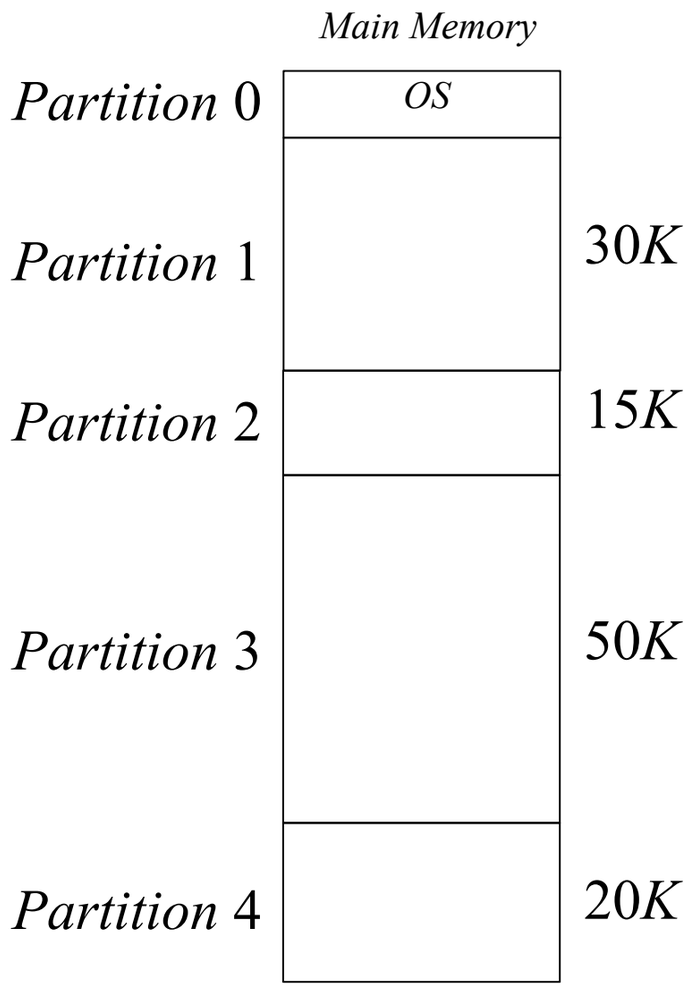
- Job *C* has to wait until a any job exit and has enough 30*K* memory space.

✔ This scheme also use the memory efficiently but slower implementation.



Example 7: Worst-Fit Allocation

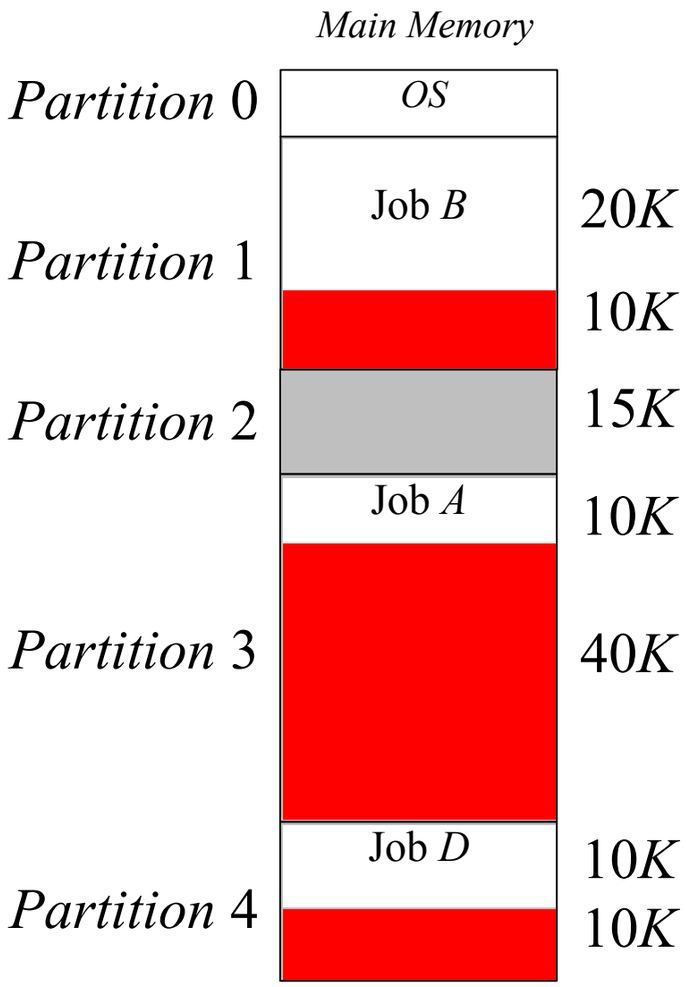
(Assume fixed partition are used)



Init. jobs:
A (10*K*)
B (20*K*)
C (30*K*)
D (10*K*)







Unused partition (15K)

Internal Fragmentation (60K)

Init. jobs:
A (10K)
B (20K)
C (30K)
D (10K)

- Job C has to wait until a large block become available, even though there is 75K unused memory space.



Best-Fit versus First-Fit Allocation

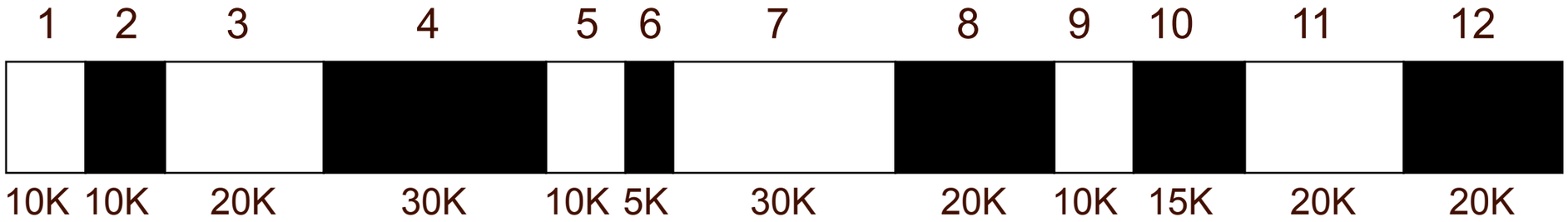
- **First-fit memory allocation**
  - Advantage: faster in making allocation
  - Disadvantage: leads to memory waste

- **Best-fit memory allocation**
  - Advantage: makes the best use of memory space
  - Disadvantage: slower in making allocation

First-fit and Best-fit better than Worst-fit in terms of speed and storage utilization.

Exercise:

Given memory is allocated as specified below:



Assume partition requests are in the following order:

P_1 – 20K, P_2 – 10K, P_3 – 5K

Determine which partition each request will be allocated if the following strategy is used:

- (i) First-fit (ii) Next-fit (iii) Best-fit (iv) Worst-fit

Answers

P1
20K
P2
10K
P3
5K
1 2 3 4 5 6 7 8 9 10 11 12
First-fit
10K 10K 20K 20K 10K 10K 5K 30K 20K 10K 15K 20K 20K
Next-fit
10K 10K 20K 20K 10K 10K 5K 30K 10K 10K 10K 5K 10K 20K 20K
Best-fit
10K 10K 20K 30K 10K 5K 30K 20K 10K 15K 20K 20K
Worst-fit
10K 10K 20K 20K 10K 10K 5K 30K 10K 10K 10K 15K 20K 5K 15K

Summary of Contiguous Memory Allocation

- Early memory management techniques
  - Fixed partitions, dynamic partitions, and relocatable dynamic partitions

- Common requirements of four memory management techniques
  - Entire program loaded into memory
  - Contiguous storage
  - Memory residency until job completed

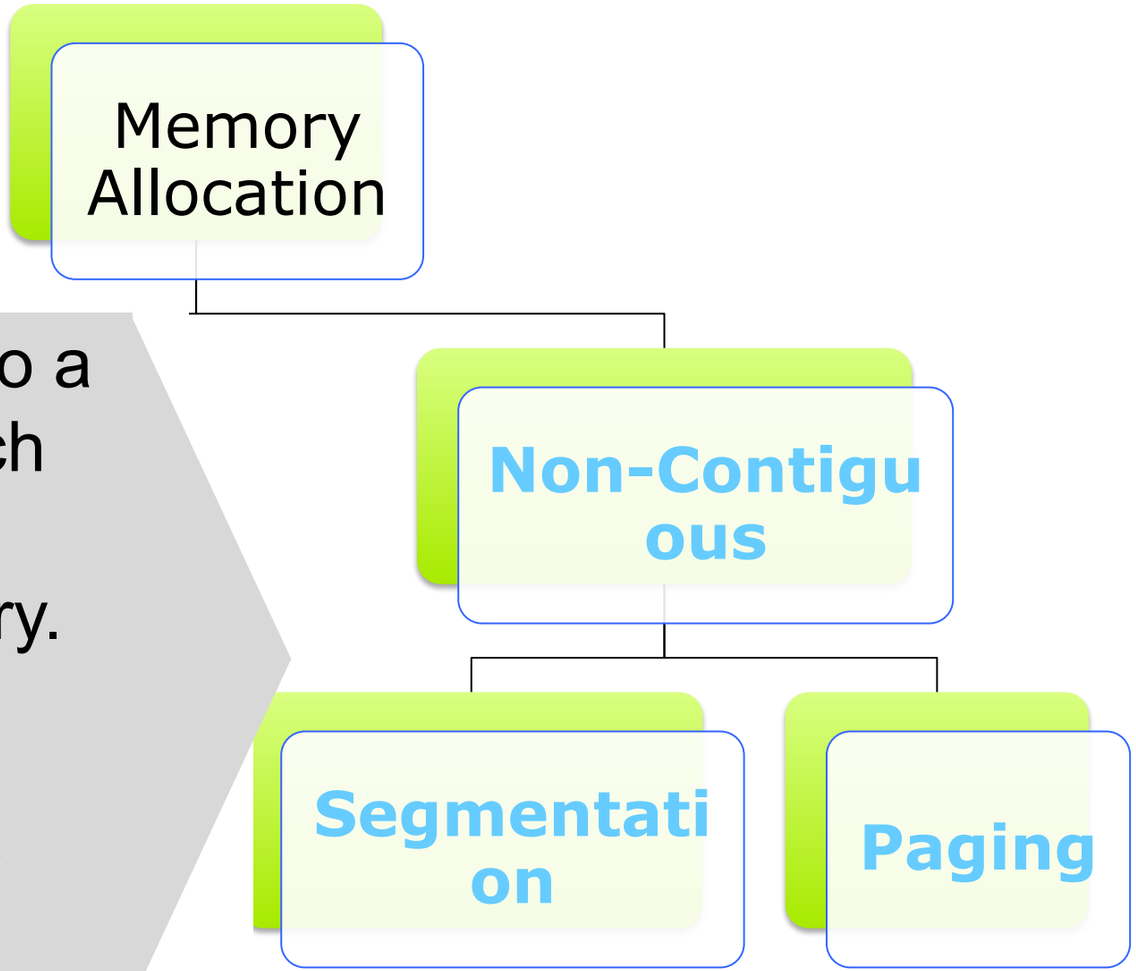
- Each places severe restrictions on job size
- Sufficient for first three generations of computers

- New modern memory management trends
  - Common characteristics of memory schemes
    - Programs are not stored in contiguous memory
    - Not all segments reside in memory during job execution



Non Contiguous Memory Allocation

- Partition the a program into a number of small units, each of which can reside in a different part of the memory.
- Need hardware support.
- Various methods to do the partitions:







Non Contiguous : Segmentation

Overview

- Memory-management scheme that supports user view of memory.
- A program is a collection of **segments**.

- A segment is a **logical unit** such as:

main program,	*global variables,*
procedure,	*common block,*
function,	*stack, local variables,*
method,	*symbol table,*
object	*arrays*



Non Contiguous : Segmentation

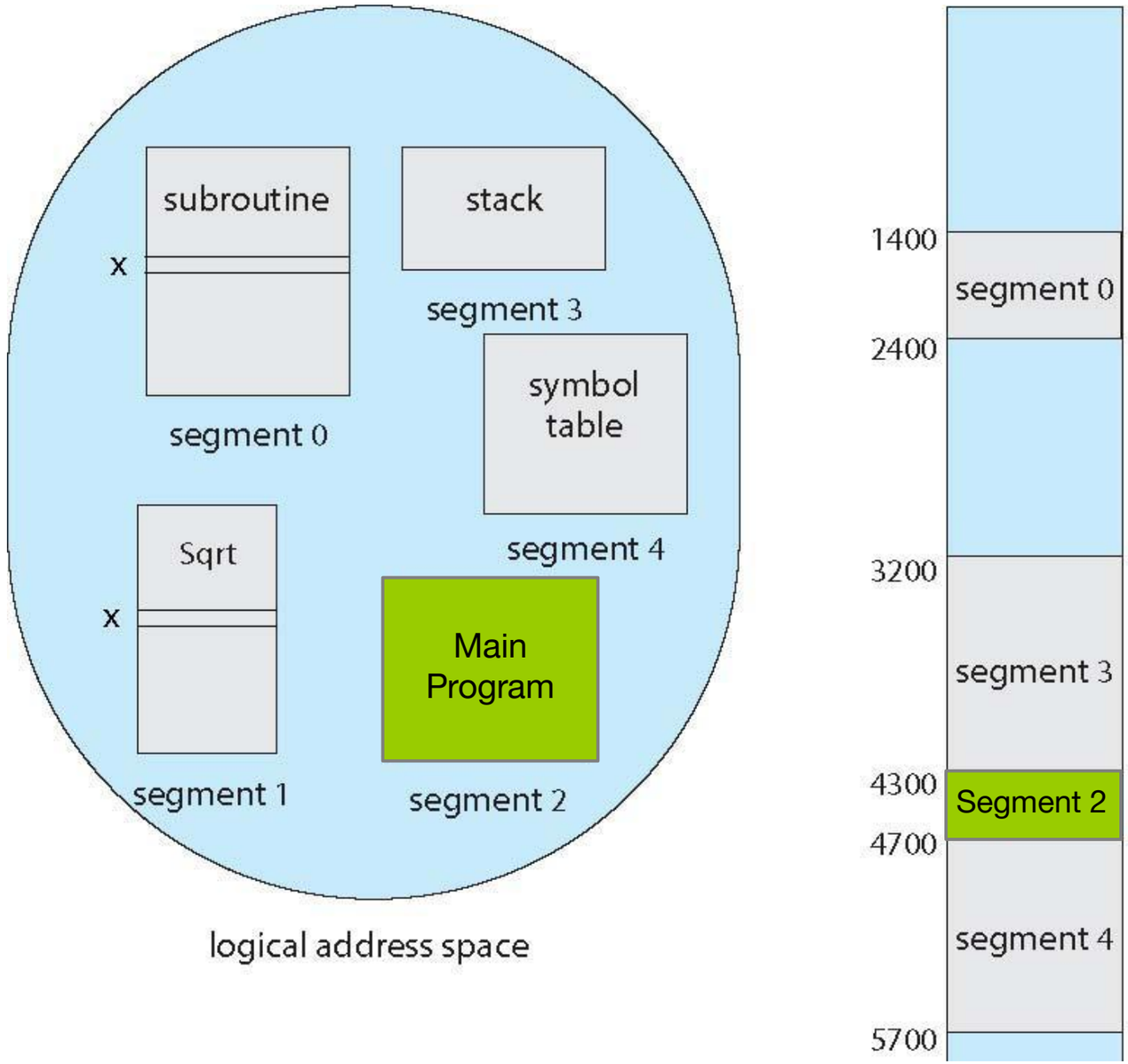


Figure 8.2 Logical and Physical Memory



Non Contiguous :Segmentation

Segmented Memory Allocation

- Each job divided into several segments.
  - Segments are different sizes.
  - One for each module containing related functions.
- Main memory is allocated dynamically.
- Program's structural modules determine segments.
  - Each segment numbered when compiled/assembled.
  - Need **Segment Table**.





Non Contiguous : Segmentation

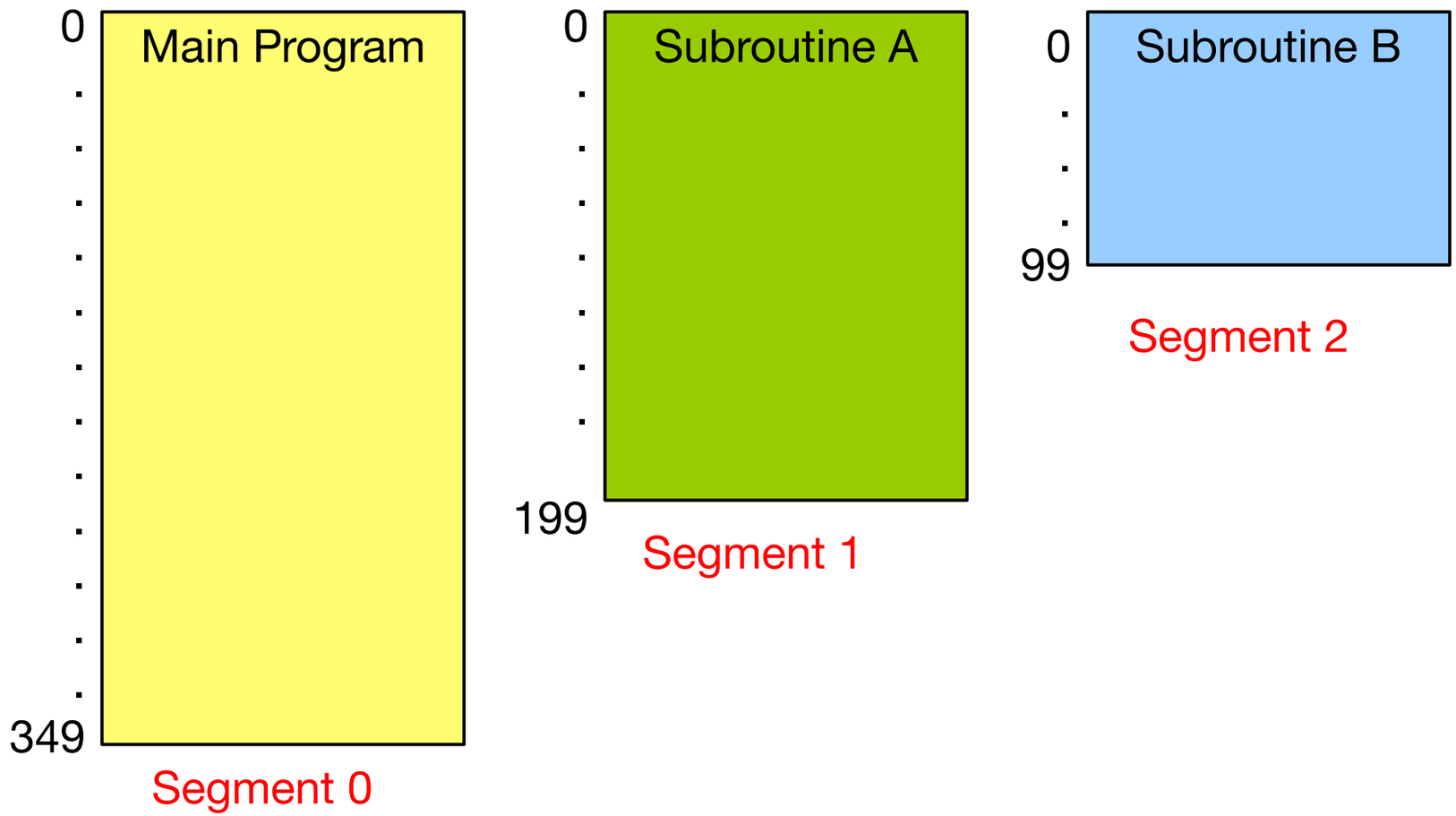
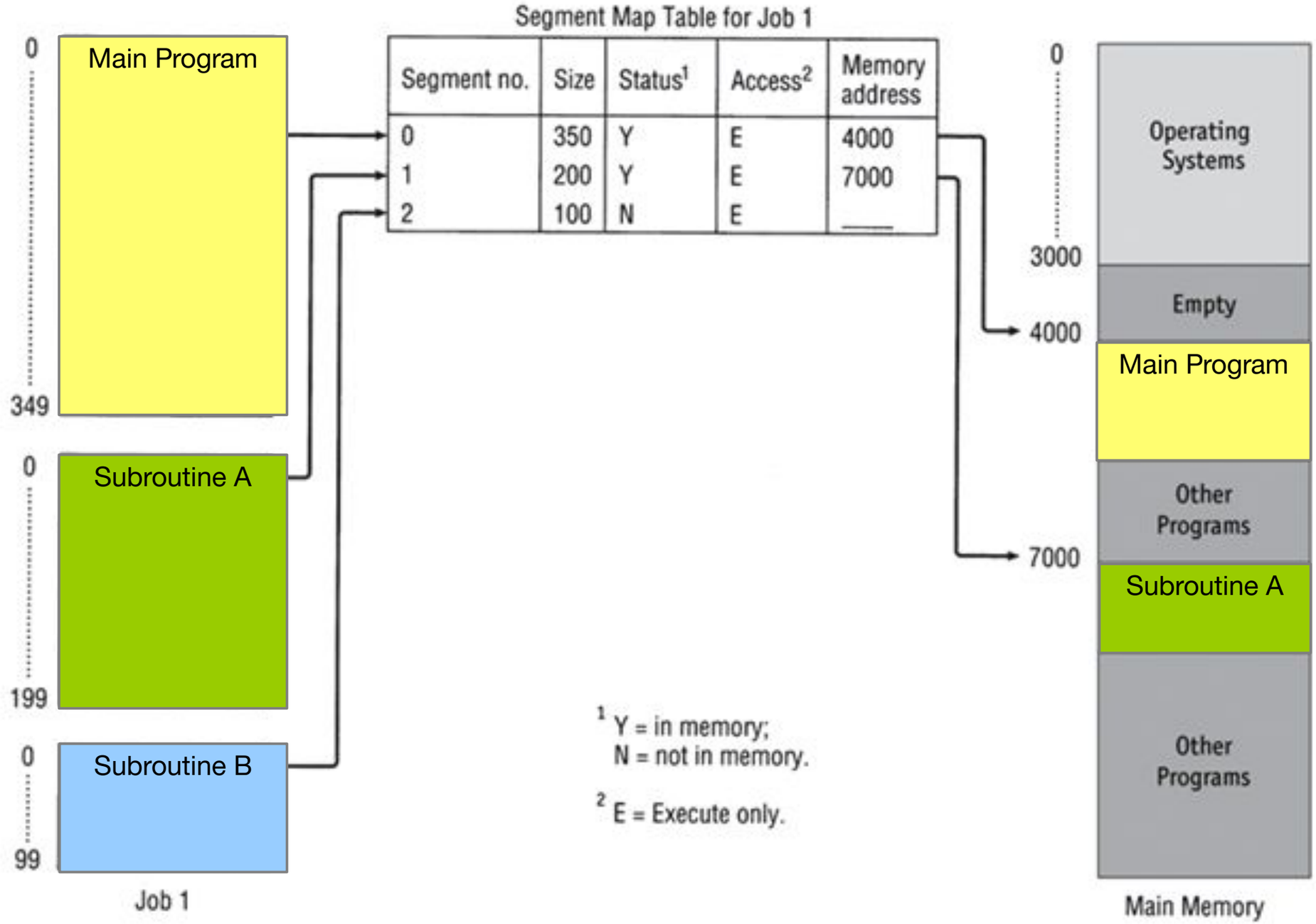


Figure Segmented memory allocation: The job includes a main program, Subroutine A, and Subroutine B. It is one job divided into three segments.



Non Contiguous : Segmentation



(figure 3.14) *The Segment Map Table tracks each segment for Job 1.*



Non Contiguous : Segmentation

- Addressing scheme require Segment number and displacement / offset;

- **Advantages**
  - Internal fragmentation is removed.
  - Memory allocated dynamically.

- **Disadvantages**
  - Difficulty managing variable-length segments in secondary storage.
  - External fragmentation.





Non Contiguous : Segmentation

Segmentation hardware

- Logical address consists of a two tuple:

  `<segment-number, offset>`

- Need to map a two-dimensional logical addresses to a one-dimensional physical address. Done via **Segment table**:
  - **base** – contains the starting physical address where the segments reside in memory
  - **limit** – specifies the length of the segment.





Non Contiguous : Segmentation

Segmentation hardware

- Segment table is kept in memory:

  - **Segment-table base register (STBR)** points to the segment table's location in memory.

  - **Segment-table length register (STLR)** indicates number of segments used by a program;

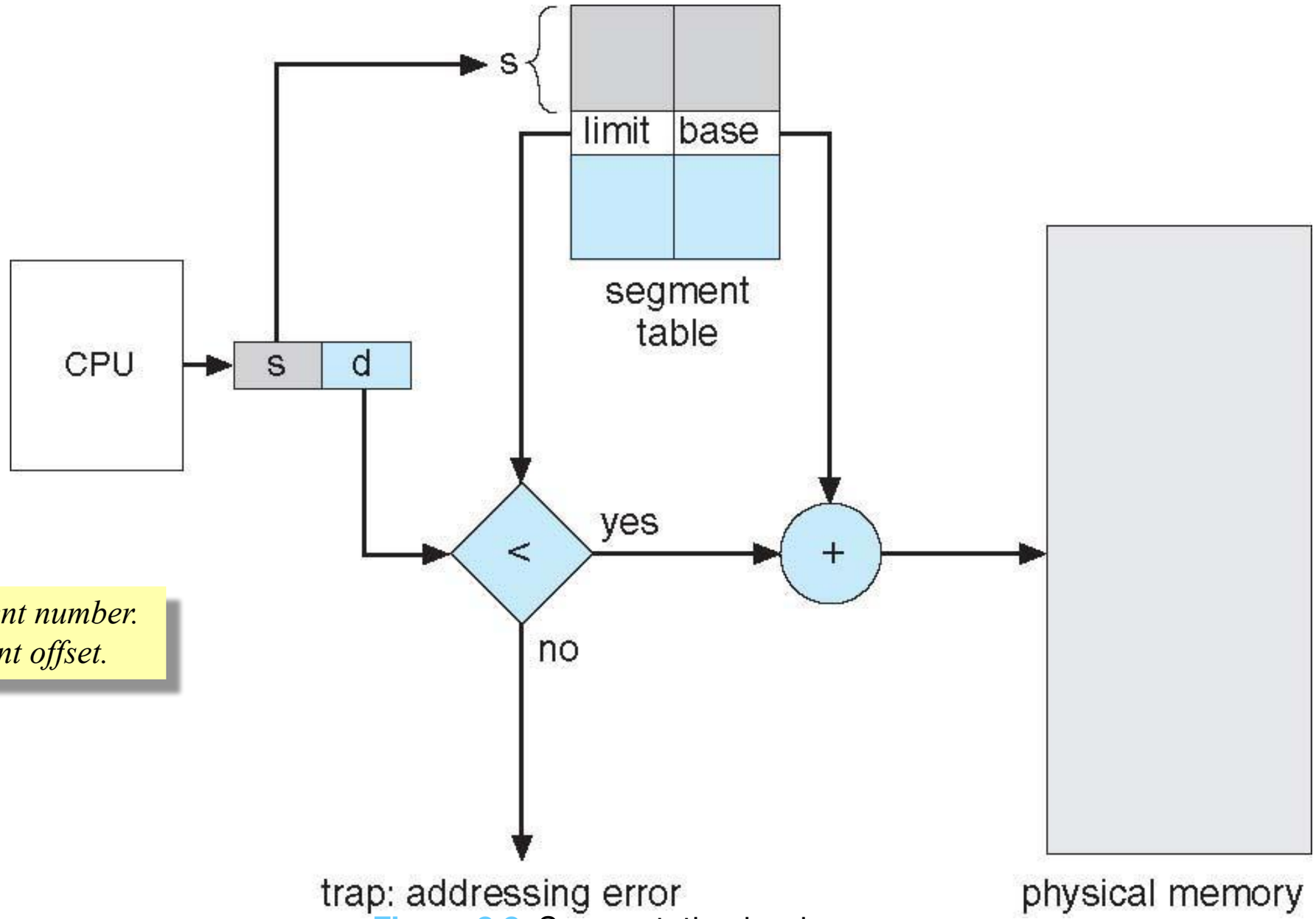
    ✔ segment number s is legal if $s < \text{STLR}$





Non Contiguous : Segmentation

Segmentation hardware



s : Segment number.
d : Segment offset.

Figure 8.8 Segmentation hardware.





Non Contiguous : Segmentation

Example

What is the physical memory address for the logical address 2 : 53?

(Page: 360)

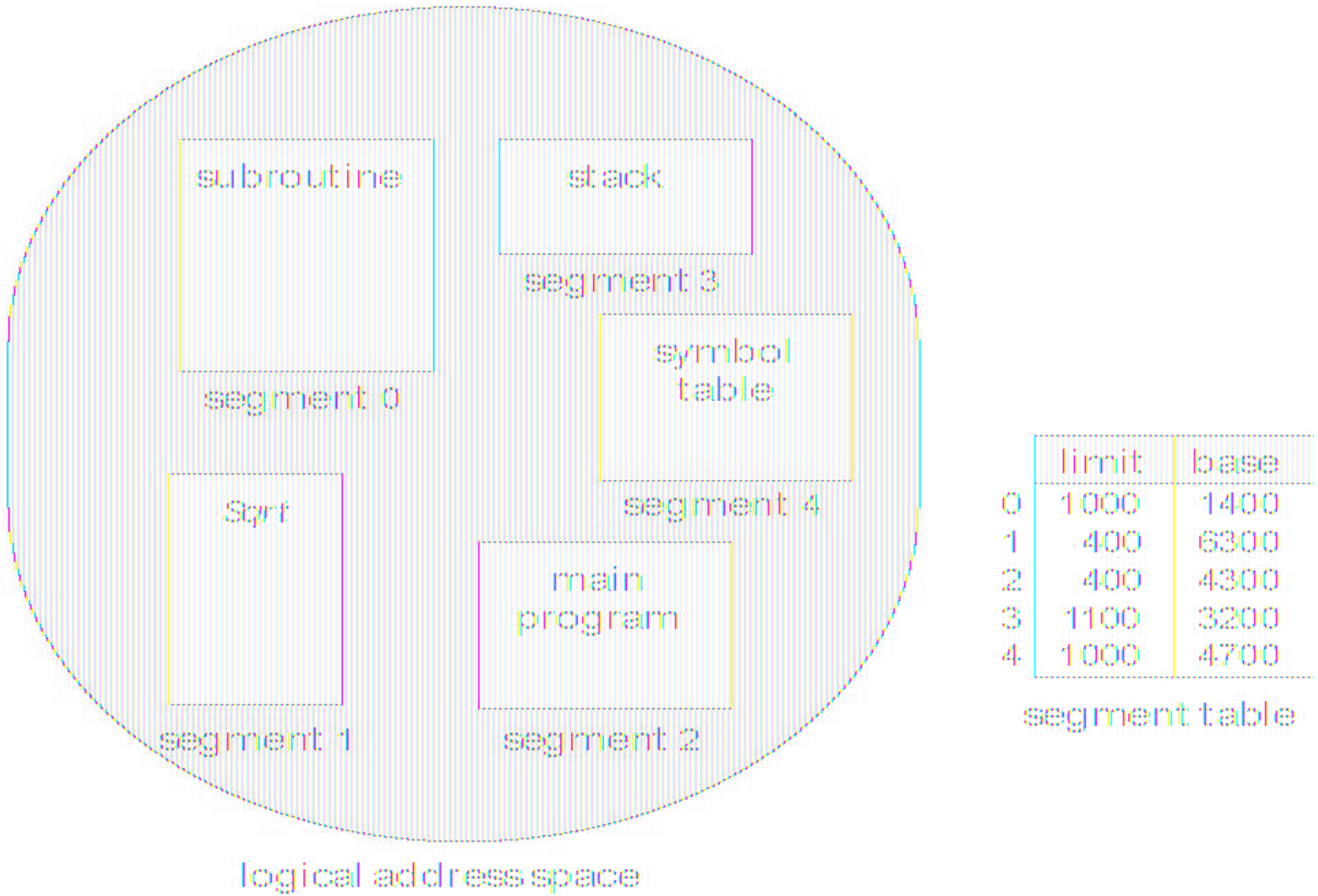


Figure 8.9 Example of segmentation.



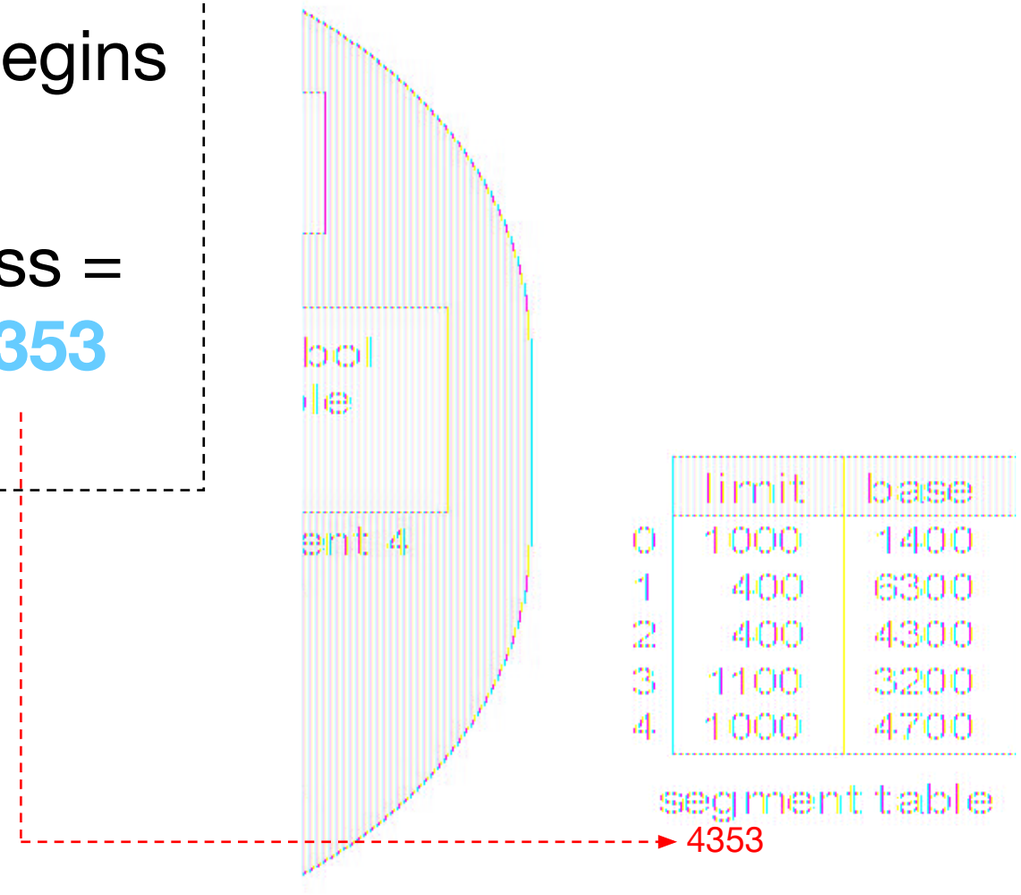


Non Contiguous : Segmentation

Solution

Segment 2 is 400 bytes long, and begins at location 4300.

- Physical address = 4300 + 53 = **4353**





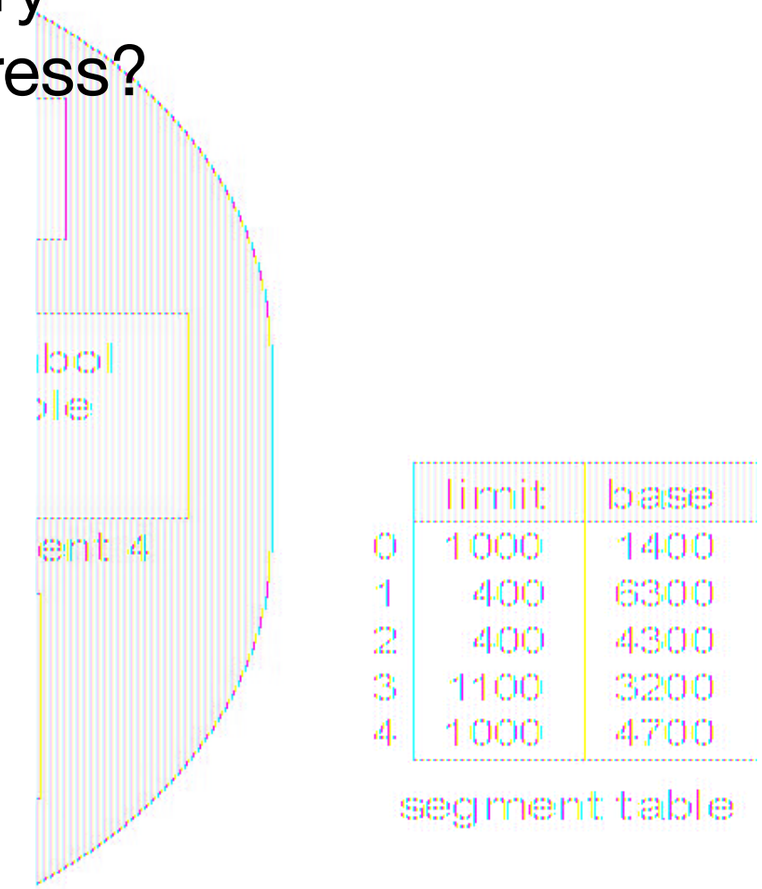
Non Contiguous : Segmentation

Exercise

What is the physical memory address for the logical address?

a) 3 : 852

b) 0 : 1222



- **Answer:**
- **A reference to segment 3, byte 852, is mapped to Physical memory address: 3200 (the base of segment 3) + 852 = 4502.**

- **A reference to byte 1222 of segment 0 would result in a trap to the OS, as this segment is only 1000 bytes long.**

(Page: 360)



Non Contiguous : Paging

Overview

- Physical address space of a process can be **non-contiguous**.

- Process is divided into **fixed-size** blocks, each of which may reside in a different part of physical memory.

- Divide physical memory into **fixed-size** blocks called **frames**:
  - Size of a frame is power of 2 between 512 Bytes and 8192 bytes (2^9 - 2^{13}).

- Divide logical memory into blocks of same size as frames called **pages**.





Non Contiguous : Paging

Overview

- Backing store, where the program is permanently residing, is also split into storage units (called **blocks**), which are the same size as the **frames** and **pages**.

- Physical memory allocated whenever the latter is available:
  - Avoids external fragmentation.
  - Still have Internal fragmentation.

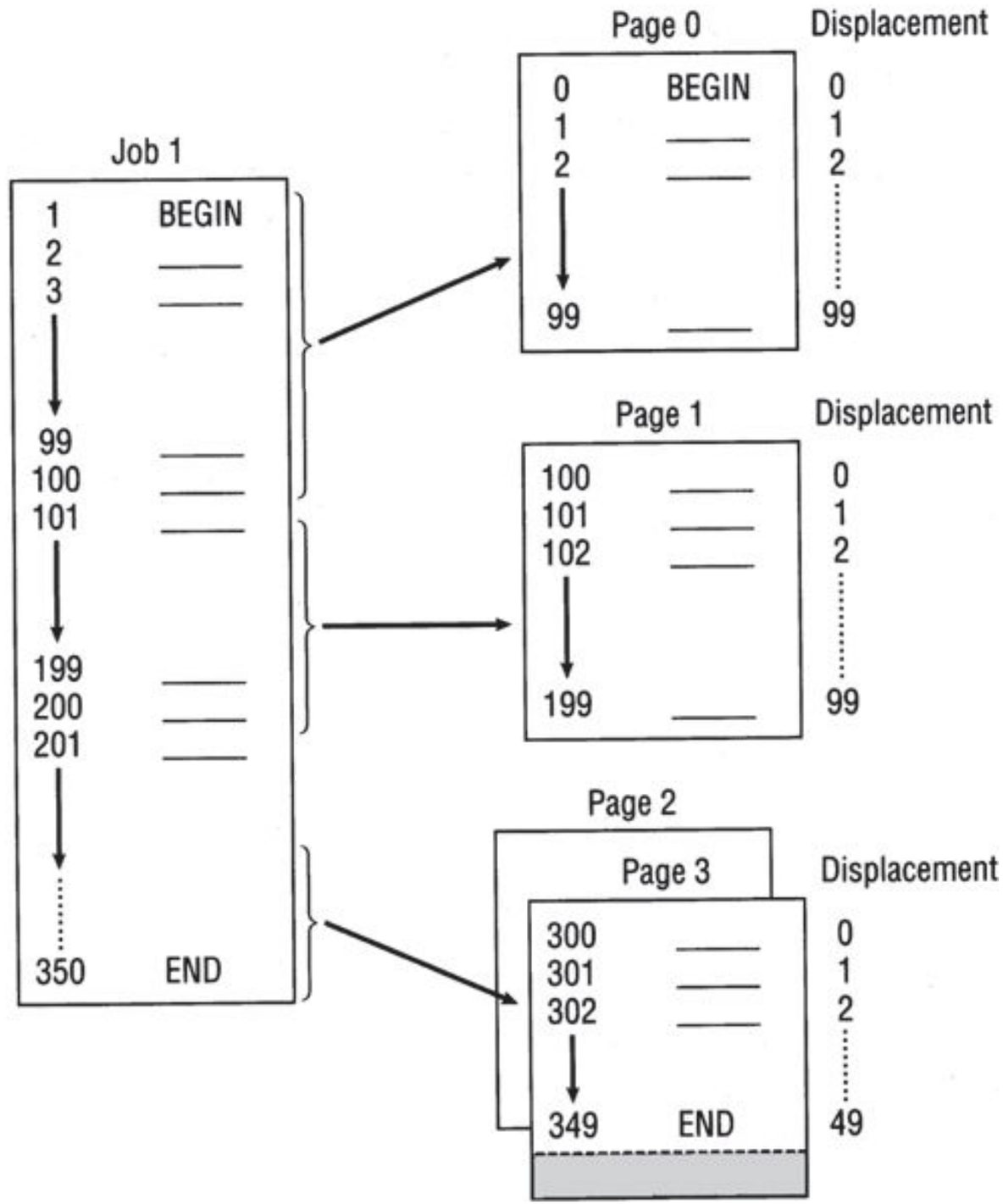




Non Contiguous : Paging

Overview

- Program is 350 lines long and is divided into 4 page frames of 100 lines each.







Non Contiguous : Paging

Overview

Consider a program with 350 lines long. Each page frame in a system can hold 100 lines. What happen in paged memory allocation?

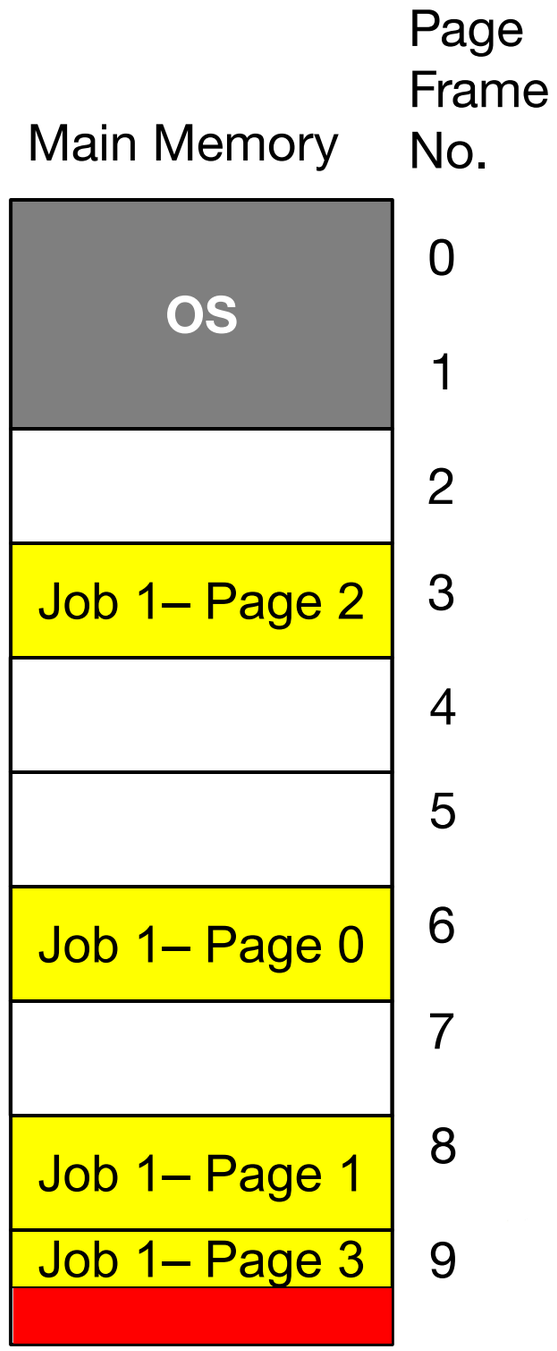
Job

100 lines	*Page 0*
100 lines	*Page 1*
100 lines	*Page 2*
50 lines	*Page 3*

Page size = Frame size

Page Table	
0	6
1	8
2	3
3	9

Page# □ Frame#



Internal Fragmentation





Non Contiguous : Paging

Overview

- **Advantages**:
  - Allows job allocation in non-contiguous memory.
  - Efficient memory use.

- **Disadvantages**:
  - Increased overhead from address resolution.
  - Internal fragmentation in last page.

- Page size selection is crucial:
  - **Too small**: generates very long page table.
  - **Too large**: excessive internal fragmentation.





Non Contiguous : Paging

Basic method

- Paging keeps track of all free frames.
- To run a program of size N pages, need to find N free frames and load program from backing store.
- Set up a **page table** to translate logical to physical addresses.
- Page table is kept in memory:
  - **Page-table base register (PTBR)** points to the page table.
  - **Page-table length register (PTLR)** indicates size of the page table.

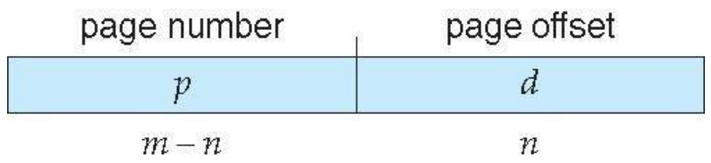




Non Contiguous : Paging

Address translation scheme

- Assume the logical address space is $2^{m.}$
- Assume page size is 2^{n}
- Address generated by CPU is divided into:
  - **Page number** (***p***) – used as an index into a **page table** which contains base address of each page in physical memory. Size of ***p*** is "m – n"
  - **Page offset** (***d***) – combined with base address to define the physical memory address that is sent to the memory unit. Size of ***d*** is "n".







Non Contiguous : Paging

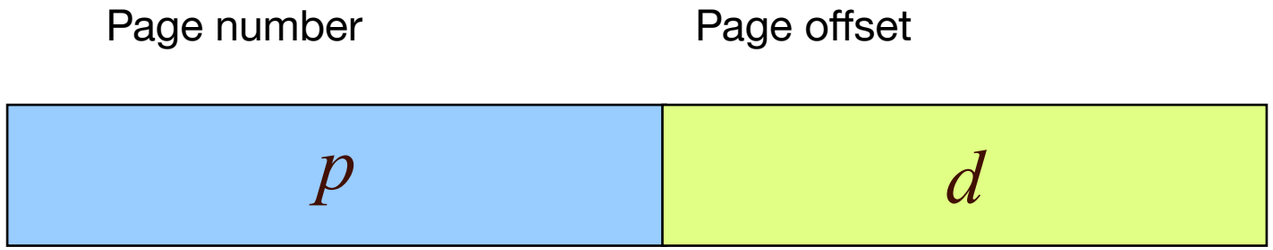
Example

Considered size of logical address is 16 MB with it page size 1 KB. Find the p bits for the page number of the address.

Solution

– size of logical address = 16 MB = 16 x 2^{20} Bytes
= 2^4 x 2^{20} = 2^{24}

– page size = 1 KB = 1024 Byte = 2^{10}



= *m – n*
= *24 – 10*
= *14 bits*

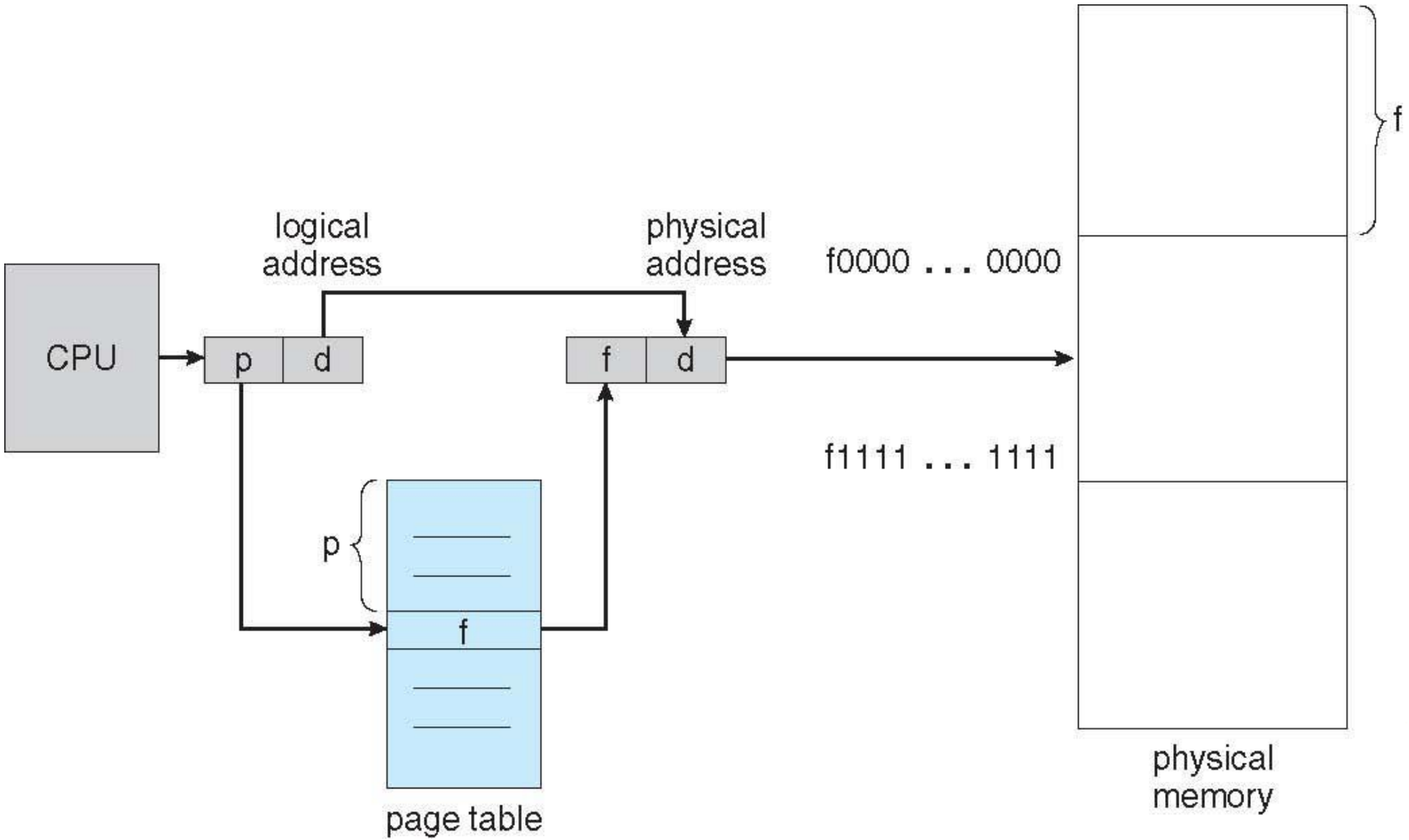
= *n*
= *10 bits*





Non Contiguous : Paging

Paging hardware

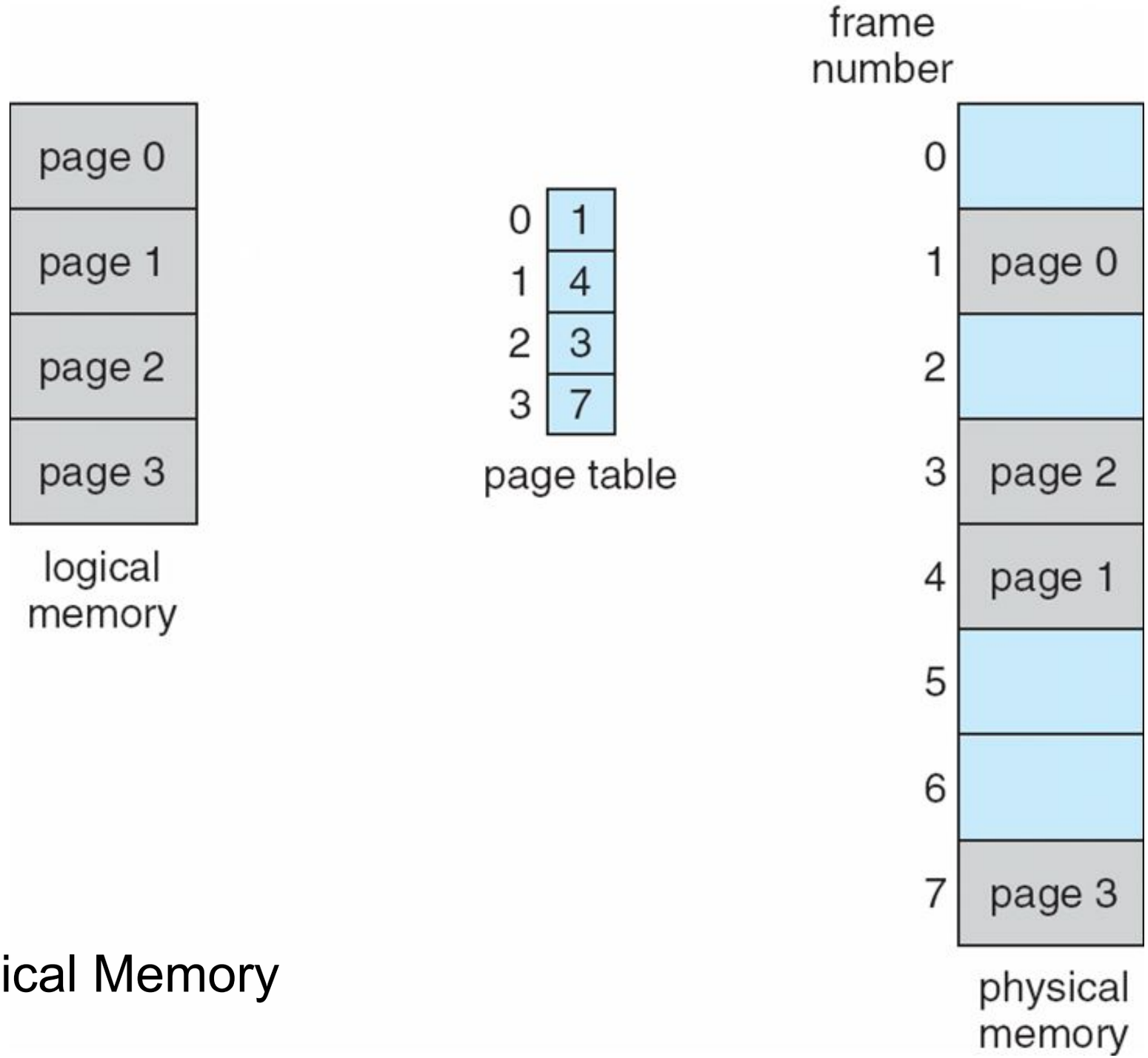






Non Contiguous : Paging

Example :



Paging Model:
Logical and Physical Memory





Non Contiguous : Paging

Example

n=2 and *m*=4

Page size = frame size = 2^n = 4-byte pages

Logical memory = Total page x Page size

= 4 x 4 = 16 Bytes = $2^m = 2^4$

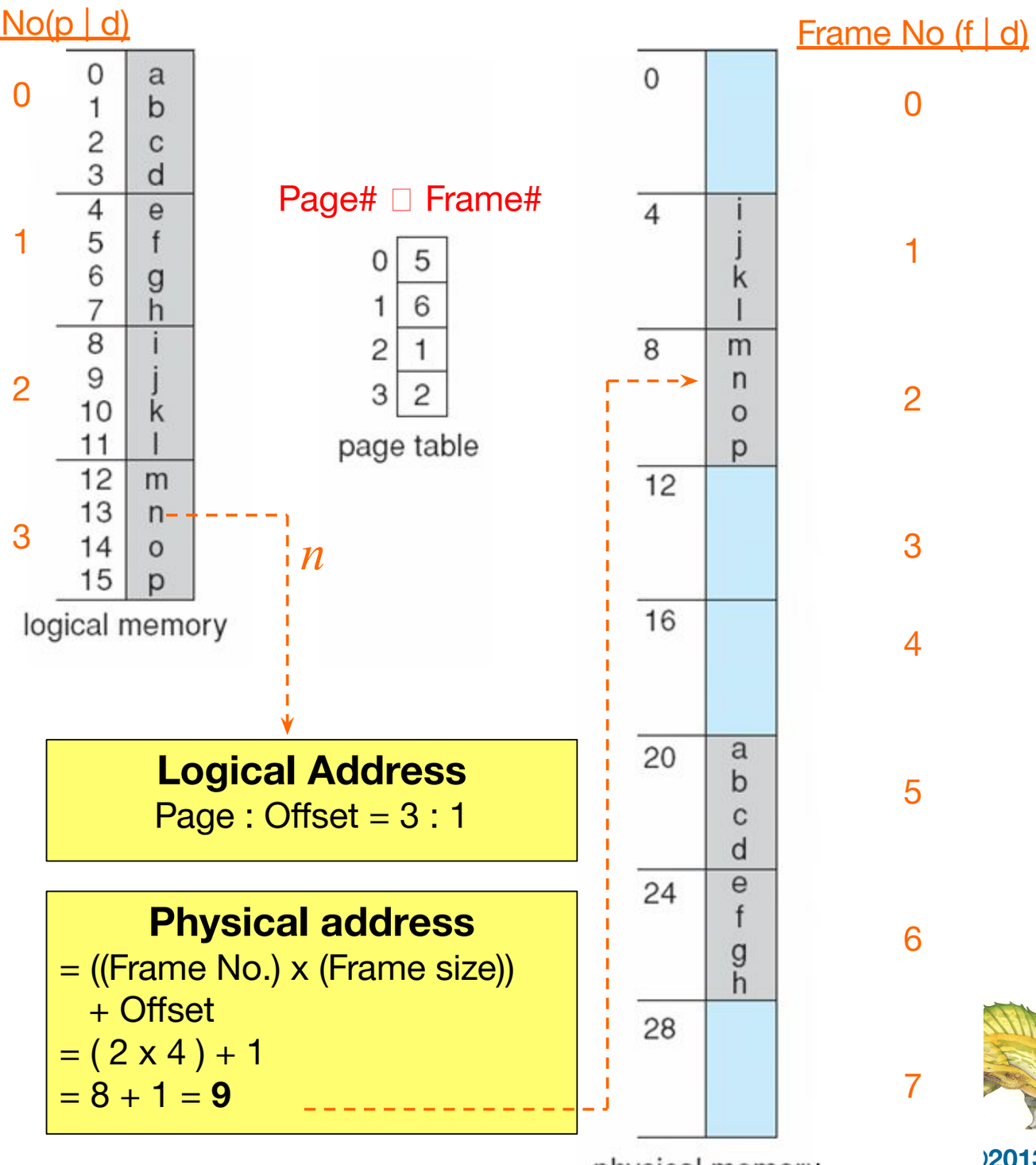
Physical memory = Total Frame x Frame size

= 8 x 4 = 32 Bytes

Physical address = (Frame No x Frame Size) + Offset

Logical address = (Page No x Page Size) + Offset

Get the physical address for *n*.





Non Contiguous : Paging

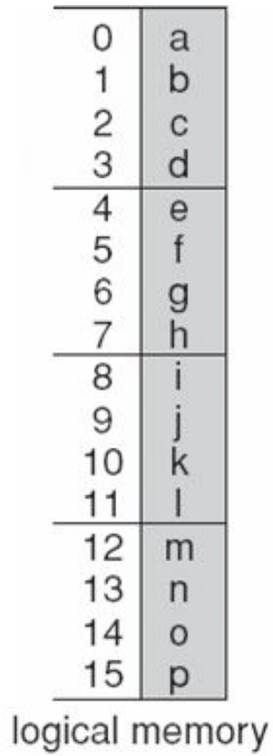
Exercise

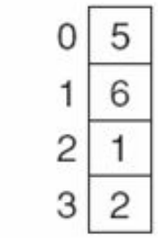
What is the physical address for the following logical addresses with given the number of the page and the offset?

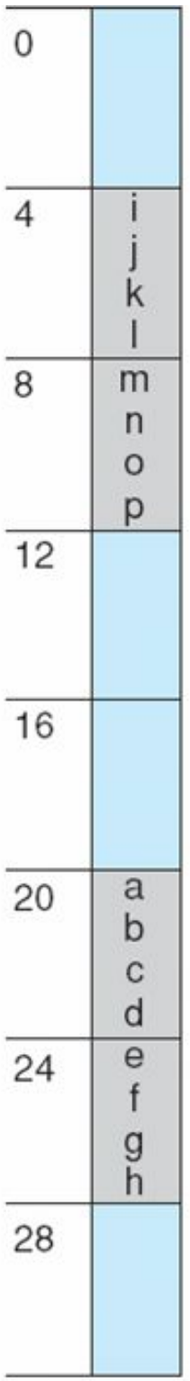
a) 0 : 3

b) 1 : 0

c) 2 : 1



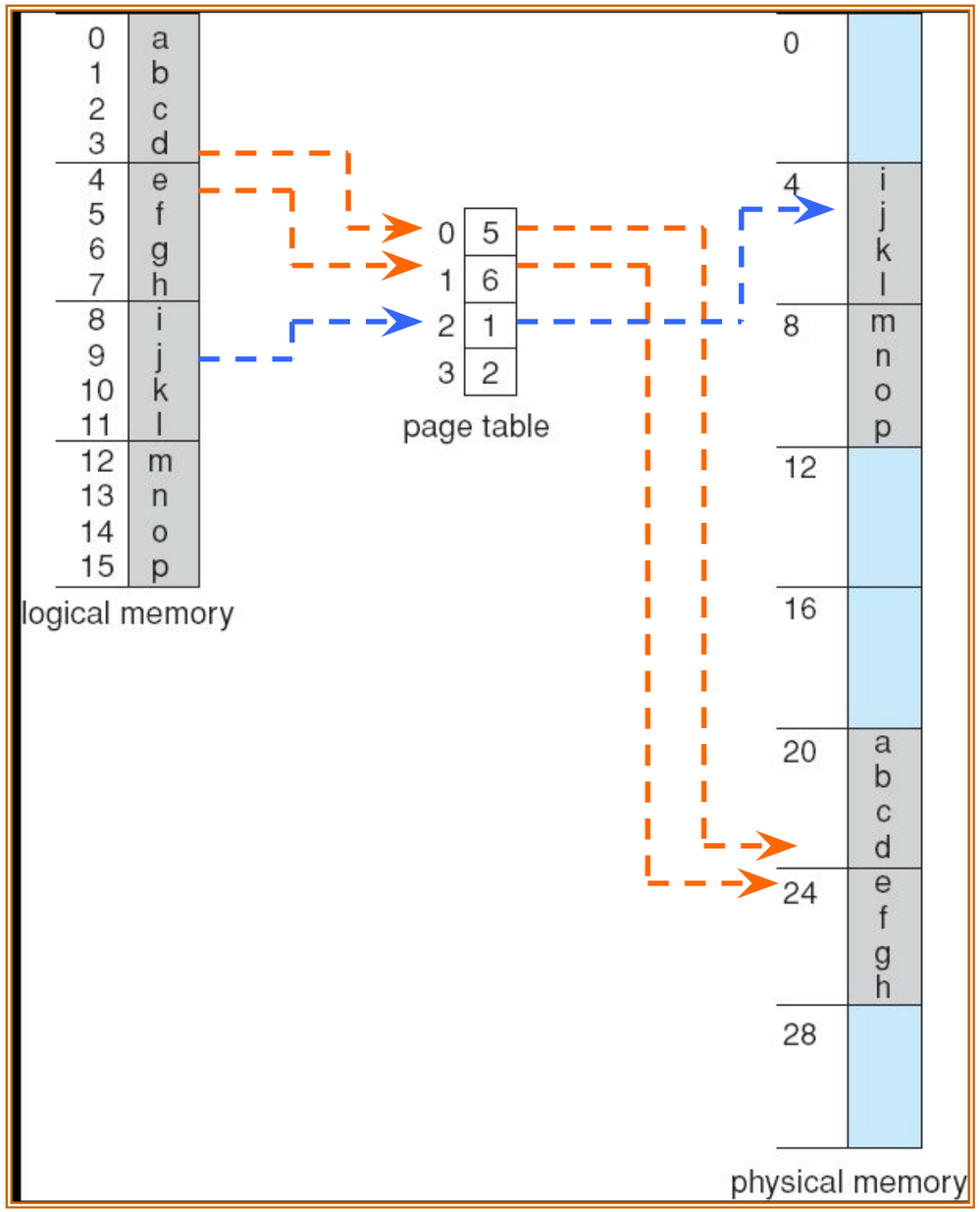








Non Contiguous : Paging



32-Byte memory and 4-Byte pages

Page	Offset
0	3

=(5 x 4) + 3
= 20 + 3
= **23**

Page	Offset
1	0

= (6 x 4) + 0
= 24 + 0
= **24**

Page	Offset
2	1

= (1 x 4) + 1
= 4 + 1
= **5**





Non Contiguous : Paging

Summary
Paging System

Logical Memory

Total size of logical memory $= 2^m$

Number of pages $= (2^m / 2^n)$

Page 0
Page 1
Page 2
. . .

Page Size = Frame Size $= 2^n$

Number of entries $= (2^m / 2^n)$

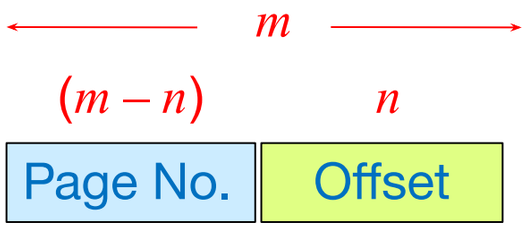
Page Table	

Physical Memory

Frame 0
Frame 1
Frame 2
. . .

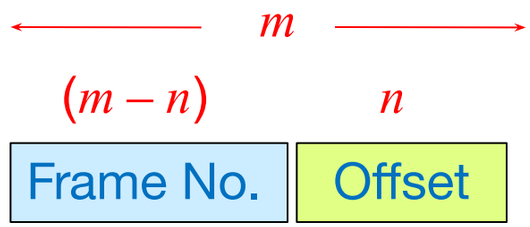
Total size of physical memory $= 2^m$

Number of frames $= (2^m / 2^n)$



p
Bit (Number of pages)

d
Bit (Size of page)



f
Bit (Number of frames)

d
Bit (Size of frame)





Non Contiguous : Paging

Exercise

Consider a paging system with 4 pages of logical address space, a page size of 4 bytes and a physical memory of 64 bytes as depicted in the following diagram.

Logical Memory

Logical address	Content
0	**A**
1	**B**
2	**C**
3	**D**
. .	. .
8	**G**
9	**H**
10	**I**
11	**J**
12	**K**
13	**L**
14	**M**
15	**N**

a) What is the frame size of the physical memory?

b) How many frames of the physical memory?

c) List the logical addresses for the following page.

Page No.	Logical Address
1	
3	





Non Contiguous : Paging

d) Calculate the page offset for logical memory location that stores the data *H*.

e) Suppose a page table for the above paging system is as follows:

0	1
1	3
2	8
3	10

Calculate the physical address in binary for data H.





Non Contiguous : Paging

Solutions:

Logical Memory

Logical address	Content
0	A
1	B
2	C
3	D
.	.
.	.
8	G
9	H
10	I
11	J
12	K
13	L
14	M
15	N

Page 0 (4 bytes)

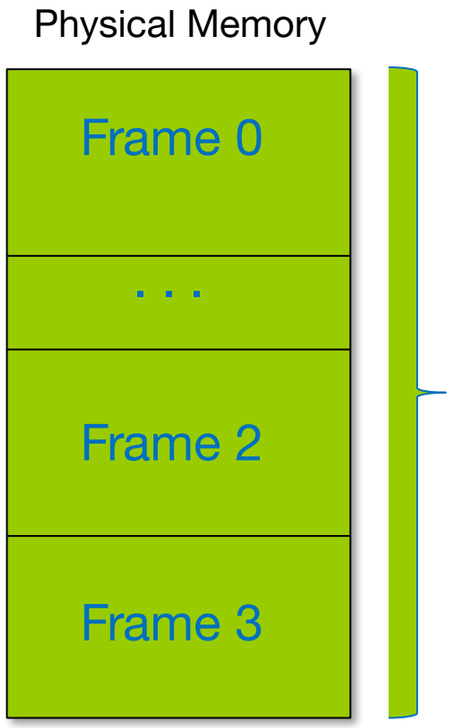
Page 1

Page 2

Page 3

Question (a):

Frame Size = Page Size = 4 bytes = 2^2



Size Physical memory = 64 bytes = 2^6

Question (b):

Number of frames = (2^6 / 2^2) = 2^4 = 16



Non Contiguous : Paging

(c) List the logical addresses for the following pages: [2 mark]

Question (c):

Page No.	Logical addresses
1	4, 5, 6, 7
2	8, 9, 10, 11

(d) Calculate the page offset for logical memory location that stores data **H**. [1 mark]

Logical Memory

Logical address	Content
0	**A**
1	**B**
2	**C**
3	**D**
.	.
.	.
8	**G**
9	**H**
10	**I**
11	**J**
12	**K**
13	**L**
14	**M**
15	**N**

Page 0

Page 1

Page 2

Page 3

Question (d):

H : logical address = 9

Logical address = (Page# x Page size) + $offset_H$

$Offset_H$ = 9 – (2 x 4)
= 9 – 8 = 1



Non Contiguous : Paging

(e) Suppose a page table for the above paging system is as follows:

0	1
1	3
2	8
3	10

Calculate the physical address in binary for data **H** [3 marks]

Logical Memory

Logical address	Content
0	**A**
1	**B**
2	**C**
3	**D**
.	.
.	.
8	**G**
9	**H**
10	**I**
11	**J**
12	**K**
13	**L**
14	**M**
15	**N**

Page 0 | Page 1 | Page 2 | Page 3

Offset: 0 1 2 3

Page Frame

0	1
1	3
2	8
3	10

Physical address :
= (Frame# x Frame size) + $offset_H$
= (8 x 4) + 1
= 32 + 1 =
= 33_{10} □ 100001_2



Non Contiguous : Paging

Exercise

Consider a paging system with 2^{10} pages of logical address space, a page size of 256 bytes and a physical memory of 2^{24} bytes. Answer the following questions. [8 *marks*]

a) What is the total size of the logical address space?

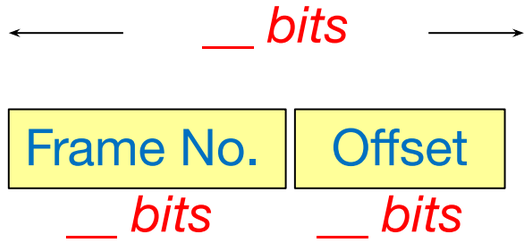
b) What is the frame size of the physical memory?

c) How many frames of the physical memory?

d) How many number of entry in the page table?

e) Calculate the offset bits for a page of virtual memory.

f) Draw the page/frame offset of the logical/physical memory address.







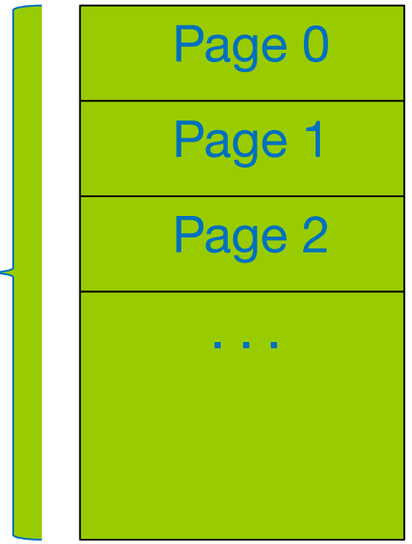
Non Contiguous : Paging

Solutions:

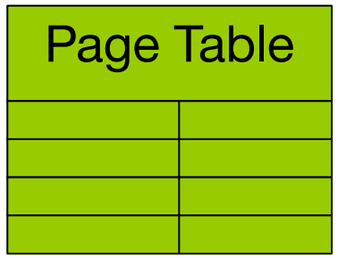
Logical Memory

Space of logical memory or number of pages = 2^{10}

Total size of logical Memory = 2^m = (No.Pages * page size) = (2^{10} * 2^8) = 2^{18}

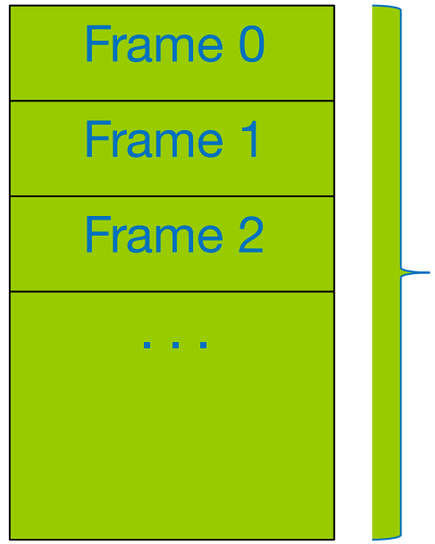


Page Size = Frame Size = 2^n = 256 bytes = 2^8



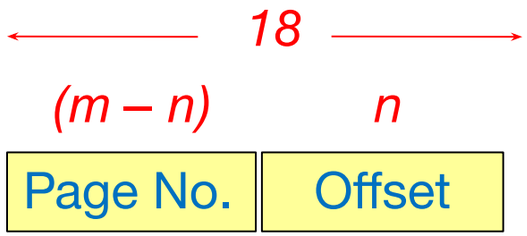
Number of entries = Number of pages = 2^{10} = 1024 entries

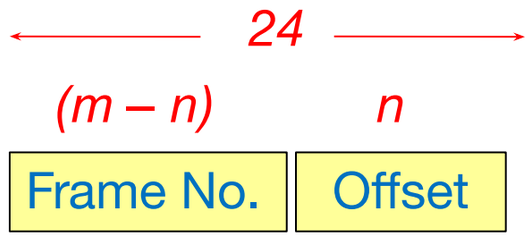
Physical Memory



Size of physical memory = 2^m = 2^{24}

Number of frames = (2^{24} / 2^8) = 2^{16}









Non Contiguous : Paging

Memory protection

- Memory protection implemented by associating **protection bits** with each frame to indicate if "*read-only*" or "*read-write*" access is allowed.
  - Can also add more bits to indicate "*execute-only*" and so on.
- **Valid-invalid** bit attached to each entry in the page table:
  - "*valid*" indicates that the associated page is in the process' logical address space, and is thus is a legal page.
  - "*invalid*" indicates that the page is not in the process' logical address space.
  - Or use **Page-Table Length Register (PTLR)**





Non Contiguous : Paging

Example

(Page: 370)

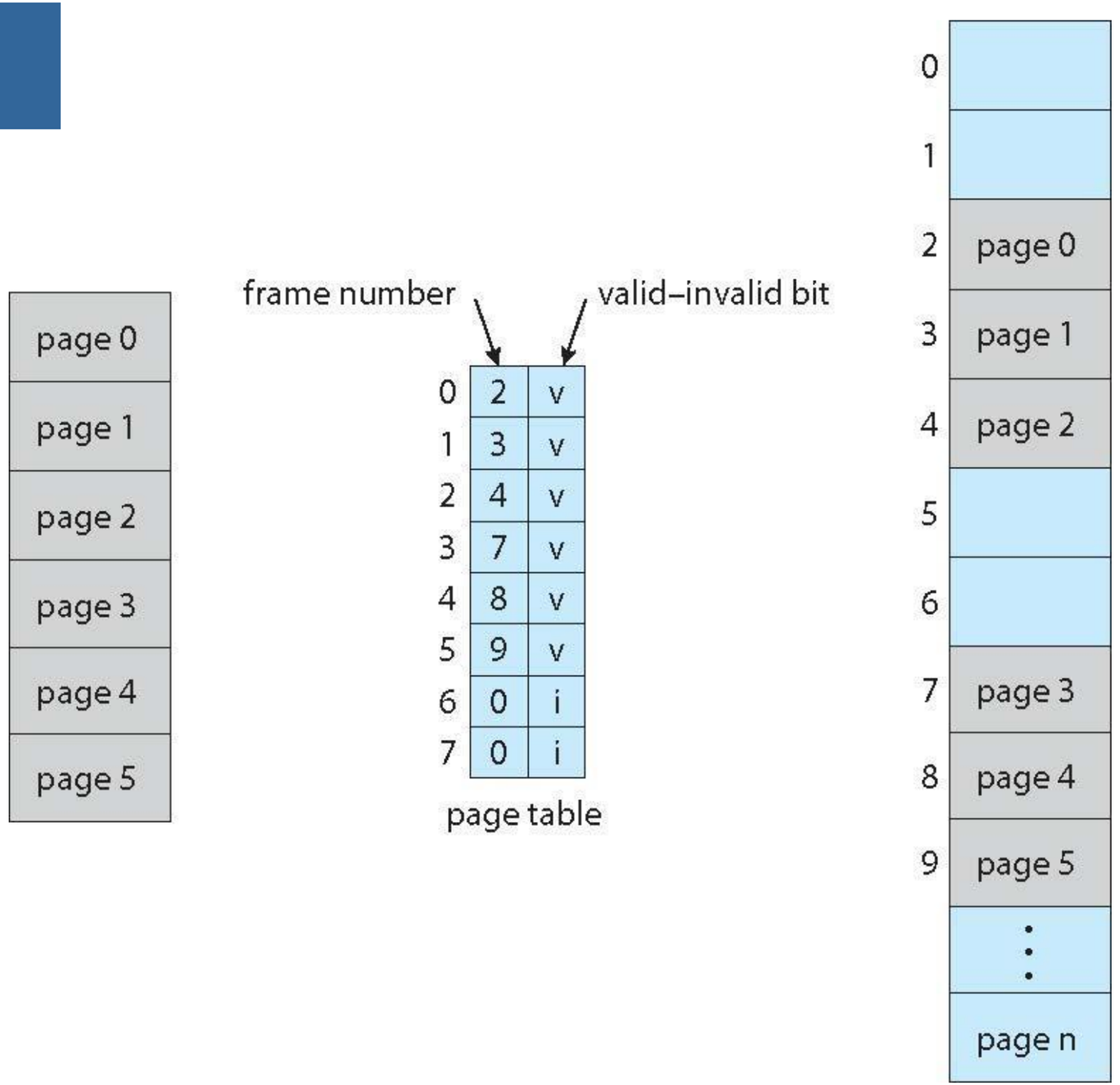


Figure 8.15 Valid (v) or Invalid (i) Bit In A Page Table.





Summary

- Memory-management algorithms for multiprogrammed operating systems range from the simple single-user system approach to segmentation and paging.

- The most important determinant of the method used is the **hardware** provided.
  - ✔ Every memory address generated by the CPU must be **checked** for legality and possibly mapped to a physical address.
  - ✔ Cannot be implemented in software.





Summary

- The various memory-management algorithms for multiprogramming differ in many aspects:
  - ✔ Hardware support
  - ✔ Performance
  - ✔ Fragmentation
  - ✔ Relocation
  - ✔ Swapping
  - ✔ Sharing
  - ✔ Protection

